Design, Implementation and Cloud Evaluation of a Fault-Tolerant, Horizontally Sharded Durable Execution Engine

A PROJECT REPORT

Submitted by

BL.EN.U4AID23007 <NAME 1>

BL.EN.U4AID23031 <NAME 2>

BL.EN.U4AID230XX <NAME 3>

BL.EN.U4AID23065 <NAME 4>

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

DECEMBER 2026

ACKNOWLEDGEMENTS

The satisfaction that accompanies the successful completion of any task would be incomplete without mention of the people who made it possible, and whose constant encouragement and guidance have been a source of inspiration throughout the course of this project work.

We offer our sincere pranams at the lotus feet of **"AMMA", MATA AMRITANANDAMAYI DEVI**, who showered her blessing upon us throughout the course of this project work.

We owe our gratitude to **Prof. Manoj P.**, Director, Amrita Vishwa Vidyapeetham Bengaluru Campus. We would like to place our heartfelt gratitude to **Dr. Gopalakrishnan E.A.**, Principal, Amrita School of Computing and Amrita School of Artificial Intelligence, Bengaluru, for his valuable support and inspiration.

It is a great pleasure to express our gratitude and indebtedness to our project guide **<GUIDE NAME AND DESIGNATION>**, Department of Computer Science and Engineering, Amrita School of Computing, Bengaluru, for the valuable guidance, encouragement, moral support and affection extended to us throughout the project work.

We express our gratitude to the project panel members for their suggestions, encouragement and moral support during the course of the project, and to all faculty members for their academic support. Finally, we are forever grateful to our parents, who have loved, supported and encouraged us in all our endeavours.

ABSTRACT

A long-running workflow, such as an order, a payment or a multi-step software agent, changes the state of several external services, and a crash part way through it either loses paid-for work or repeats work that must not happen twice. Durable execution engines prevent this by journalling every workflow decision and recovering by deterministic replay, but production engines depend on four to six infrastructure services. This report presents *durexec*, a durable execution engine in Go whose only dependency is PostgreSQL. Workflows are hashed to a fixed set of shards; each shard is owned through a lease held in a database row, and a fencing token re-checked inside every write transaction stops a node that has lost its lease from committing. A journalled semantic check lets the engine recognise a reworded retry of an action already taken by a language-model agent. On Amazon EKS with three engine nodes and one PostgreSQL host, a closed-loop workload reached 131 workflows per second at a 99th-percentile latency of 3.1 s, limited by database CPU, and the sustainable open-loop rate was about 100 workflows per second. Workers, shards and engine replicas changed throughput by at most 19%, and extra replicas gave no gain while the database was the bottleneck. In sixteen fault-injection trials, covering engine termination, a 35 s process freeze, loss of database connections, a node drain and a database reboot, every accepted workflow completed, no journal invariant was violated and every write by a stale owner was rejected.

Keywords: durable execution, event sourcing, deterministic replay, leases, fencing tokens, sharding, fault tolerance, PostgreSQL.

TABLE OF CONTENTS

Page No.

ACKNOWLEDGEMENTS i

ABSTRACT ii

LIST OF FIGURES v

LIST OF TABLES vii

CHAPTER 1 – INTRODUCTION 1

1.1 Background 1

1.2 Motivation 3

1.3 Problem Statement 4

1.4 Objectives 4

1.5 Scope and Limitations. 4

1.6 Contributions 5

1.7 Organisation of the Report 6

CHAPTER 2 – LITERATURE SURVEY 7

2.1 Durable Execution and Workflow Correctness 7

2.2 Sharding and Partition Management 8

2.3 Ownership, Coordination and Fencing 10

2.4 The Saga Pattern and Compensation 11

2.5 Failure Modes of LLM Agent Systems 13

2.6 Semantic Caching and Embedding-Based Similarity 14

2.7 Comparative Synthesis 15

2.8 Identified Research Gap. 16

CHAPTER 3 – SYSTEM REQUIREMENTS AND ANALYSIS 19

3.1 Functional Requirements 19

3.2 Non-Functional Requirements 19

3.3 Software Requirements 23

3.4 Hardware Requirements 25
3.5 Assumptions and Constraints 25
3.6 Analysis of the Chosen Approach 28
CHAPTER 4 – SYSTEM DESIGN 30
4.1 High Level Design 30
4.2 Low Level Design 41
REFERENCES 65

LIST OF FIGURES

2.1 Positioning of the surveyed systems on the two axes this project cares about. Systems in the upper half require a coordination service outside their data store; systems in the right half partition execution across machines. The lower-right quadrant — partitioned without a coordination service — is the target of this work. 12

2.2 The crash-induced, semantically duplicate side effect that exact-match idempotency cannot detect. Completed steps resolve from history and are not re-run, but a planning step that re-executes at a non-zero sampling temperature re-issues the same intent in different words, producing a hash that matches no recorded key. 16

3.1 The container topology against which the requirements of this chapter are sized, and on which every measurement reported by this project was taken. Three engine nodes share one PostgreSQL leader with a streaming standby; the embedder and the external-effect simulator are test-only services and are never exposed beyond the host. 26

4.1 The five architectural tiers of the engine. Every node runs the client-facing gateway, the execution tier and the control tier in one process; only the persistence tier is shared. The control tier is where every mutating path is gated by a fencing-token check, which is what makes partitioned execution safe without a coordination service. 32

4.2 The two-level mapping. The first mapping is fixed for the lifetime of a workflow; only the second changes when the cluster is resized. This is what reduces rebalancing from a data migration to an ownership handoff. 35

4.3 Why the lease alone is insufficient and the in-transaction token check is what provides safety. Without fencing, a node that was frozen past its lease expiry commits alongside the new owner and produces split-brain. With fencing, it commits nothing. 38

4.4 The six tables holding every piece of durable state in the system. Every table carries `shard_id`, which is what sharding physically is in this design; no statement is ever issued without it. 42

4.5 The workflow task loop. Step 8 is a single transaction, which is what makes the system safe; step 9 discards the in-memory state, which is why correctness never depends on retaining it. 46

4.6 A worked trace across three tasks. The crucial moment is the writing of sequence 5: the *intent* to charge is durably recorded before any money moves, which is the entire basis of the engine's safety. 50

4.7 Every place a crash can land during one activity. Three of the four windows recover safely without intervention. Window C is irreducible, and the design says so rather than claiming exactly-once execution. . . 54

4.8 Saga compensation. The stack exists only in memory and is rebuilt by replay; compensations run in reverse order as ordinary activities; and a compensation that exhausts its retries leaves the workflow in a terminal state requiring human intervention. 57

4.9 The semantic idempotency decision path. The non-deterministic verdict is computed once in new mode and journalled; every subsequent replay reads the recorded verdict and performs no embedding call, no similarity computation and no network access. 59

4.10 The goroutine composition of one node. Worker counts are per owned shard, so a node's concurrency scales with the number of shards it owns rather than with cluster size. 63

LIST OF TABLES

1.1 Terminology used in this report. 2

1.2 Project objectives 5

2.1 Research on durable execution and workflow correctness 9

2.2 Production workflow orchestrators and the infrastructure each requires, from their own deployment documentation 10

2.3 Partitioning, ownership and compensation literature 13

2.4 Literature establishing that agent step repetition is a real, measured failure mode 14

2.5 Semantic caching and embedding-similarity literature 15

2.6 Positioning of this work against the closest prior art 18

3.1 Functional requirements 20

3.2 Functional requirements (continued) 21

3.3 Non-functional requirements 22

3.4 Software requirements 24

3.5 Hardware requirements of the development and measurement environment 25

3.6 Assumptions and constraints under which the design was made. 27

3.7 Architectural alternatives considered and reasons for rejection 29

4.1 Architectural tiers, their responsibilities and their state 33

4.2 The contract between workflow code and activity code 34

4.3 Partitioning parameters and the reasoning behind each default 36

4.4 Event types written to the log, grouped by the mechanism that produces them 45

4.5 Constructs forbidden in workflow code, and their replacements 49

4.6 The events written when the first workflow task commits 50
4.7 Crash-window analysis for a single activity execution 54
4.8 Retry policy parameters, evaluated by the workflow worker on seeing an activity failure 55
4.9 Failure modes of the semantic comparator. The asymmetry between them is the design's central constraint. 61
4.10 Runtime configuration parameters and their defaults 64

CHAPTER - 1

INTRODUCTION

1.1 Background

Much of the software that runs a business is not a single request answered in milliseconds but a process that unfolds over seconds, hours or days. Placing a food order charges a card, reserves stock, notifies a restaurant and assigns a driver. Settling an invoice debits one ledger, credits another and files a tax record. An agent built on a large language model that is asked to arrange travel plans an itinerary, calls a booking service, waits for a person to approve the result and then confirms it. Each of these is a *workflow*: a sequence of steps, spread over several independent services, in which every step depends on the results of the steps before it and most steps change the state of some system outside the workflow itself.

Two properties make workflows hard to run reliably. A workflow is stateful, because step four only makes sense in the light of what steps one to three decided. Its steps also have external side effects that the workflow cannot withdraw: once a payment provider has moved money, no local rollback can move it back. A database transaction has neither difficulty, since it is short enough to hold its locks throughout and its effects are confined to a system that can undo them. A workflow has both.

As a result, an ordinary crash becomes a correctness problem. Suppose the order workflow is interrupted after the card has been charged and the stock reserved, but before the restaurant has been told. A system that has kept no durable record of its progress has two choices when it restarts. It can run the workflow again from the beginning, charging the card a second time, or it can abandon the workflow, keeping the customer's money for an order that will never be delivered. Without a record of what was already done, there is no correct third option.

The usual responses in practice are retry loops, periodic jobs that poll a status column, and message queues whose consumers record progress in application tables. Each works for the workflow it was written for, and each must be written again for the next one.

Durable execution is the name given to systems that solve the problem once for every workflow built on them [1, 2]. The idea on which they rest, and on which this report rests, is that *every decision a workflow makes is appended to a log before it is acted on, and recovery is performed by running the workflow's own code again against that log*. Completed steps return their recorded results instead of executing a second time, so the re-executed code arrives at the point the failed process had reached. Because nothing private to one machine is part of the workflow's state, the recovery can take place on a different machine.

Table 1.1 fixes the terms used throughout the report.

Table 1.1: Terminology used in this report

Term	Meaning
Workflow	Orchestration code that decides what happens next and performs no input or output itself. It must be deterministic, because it is re-executed on every recovery.
Activity	One step with an external side effect, such as charging a card or calling an API. It may be non-deterministic and runs at least once.
Event	An immutable record appended to the log describing something that has already happened.
History	The ordered events of one workflow run; the complete state of that run.
Replay	Re-executing a workflow against its history so that completed steps resolve from the log.
Shard	One of a fixed number of partitions. A workflow belongs to one shard for its whole life.
Lease	A time-limited claim on a shard, held in a database row and renewed periodically, that makes one node the owner of the shard.
Fencing token	A counter that increases whenever ownership of a shard changes; a write is accepted only with the current value.
Saga	A multi-step transaction that, if it fails part way, is undone by running a compensating action for each step that had succeeded.

Industry has adopted this approach widely. Uber's Cadence and its successor Temporal run very large numbers of workflows for ride and delivery systems [1]; Microsoft's Durable Functions provide the same model for serverless code [3]; and the DBOS project argues that durable execution belongs inside the database rather than in a separate service [4, 5]. The internal mechanisms of such an engine, namely how the log is partitioned, how state is reconstructed deterministically and how ownership of a par-

tition moves safely between machines when one fails, are less often built from first principles. This project builds one, and evaluates it on a cloud cluster under load and under injected failures.

1.2 Motivation

1.2.1 Existing engines depend on several services

The production engines surveyed for this work each require four to six cooperating infrastructure components. Netflix's Conductor uses a database, a search cluster, a queue, a server tier and a worker fleet. Temporal runs four internal services over a database and, for visibility queries, a search cluster [1]. Meta's Shard Manager, which assigns partitions across very large fleets, relies on ZooKeeper for coordination [6, 7]. Each additional component is a separate failure domain that must be configured, monitored and understood during an incident, and for a small deployment the orchestration system can cost more to operate than the application it serves. This project asks whether the same guarantees can be provided with a single dependency.

1.2.2 Partitioning and simplicity are usually traded off

The simple systems are not partitioned and the partitioned systems are not simple. DBOS obtains durable execution with very few moving parts but runs on one node [4]; Temporal and Shard Manager partition work across machines but add a coordination service or a multi-service control plane to decide which machine owns what. Our position is that the two need not be in tension, because a relational database already provides the linearizable compare-and-swap that partition ownership requires [8, 9]. What is needed is a careful use of one row per shard rather than a consensus cluster [10].

1.2.3 Agents defeat byte-level deduplication

Durable execution engines prevent duplicate side effects by recognising a repeated request, and they recognise it by comparing bytes: a hash, an idempotency key or an exact match on recorded arguments. This is sound when the arguments are fixed at the moment a step is first scheduled. It is not sound when the caller is a language-model agent, which after a crash *re-plans* rather than replays: an instruction recorded as "BLR→DEL Friday morning" may be issued again as "Bangalore to Delhi, Friday

AM”. The two strings hash differently and the flight is booked twice. Step repetition of this kind appears as a recurring category in empirical studies of multi-agent failures [11] and in Microsoft’s taxonomy of agentic failure modes [12].

1.3 Problem Statement

The problem addressed in this work is to design and implement a horizontally sharded durable execution engine that runs long, multi-step workflows to completion despite process crashes, node failures and partial failures, using event sourcing, deterministic replay, idempotent activity execution and lease-based ownership; that is deployable on a small cluster with PostgreSQL as its only infrastructure dependency; that detects semantically equivalent repeated actions issued by a recovering agent before the side effect occurs a second time; and to measure its throughput, its sensitivity to its main configuration parameters, and its recovery from injected faults on real cloud infrastructure.

Two limits are part of the statement. The engine targets correctness and fault tolerance at a moderate, measurable scale rather than the throughput of large production deployments. Ownership is obtained from a lease in a linearizable database rather than from a consensus protocol, which makes the database the availability floor of the system.

1.4 Objectives

The work has eight objectives, listed in Table 1.2 with the mechanism that realises each and the chapter in which it is evaluated.

1.5 Scope and Limitations

The scope is the engine, two demonstration applications built on it, and the test bed used to evaluate it. The main limitations are stated here so that the design and the results can be read with them in mind.

1. The engine provides at-least-once execution of activities with strong deduplication, not exactly-once execution. A crash after a side effect has happened but before it has been recorded cannot be told apart from a crash before the side effect. The engine supplies a stable deduplication key that the downstream service can use to close this window.

Table 1.2: Project objectives

No.	Objective	Mechanism	Evaluated
1	Record every state transition as an immutable event so that a run can be reconstructed exactly	Append-only event log	Ch. 6, 7
2	Resume a failed run from its history rather than from the start	Deterministic replay	Ch. 6, 7
3	Never repeat an external side effect across retries	Stable deduplication keys	Ch. 6
4	Partition execution across nodes	Fixed shards, consistent-hash ring	Ch. 7
5	Lose no progress when a shard owner fails	Leases with fencing tokens	Ch. 7
6	Undo multi-step transactions that fail part way	Saga compensation	Ch. 6
7	Detect reworded duplicate agent actions	Journalled semantic verdict	Ch. 6
8	Measure capacity, sensitivity and recovery on cloud hardware	EKS test bed, fault injector	Ch. 7

2. Ownership relies on PostgreSQL as the single source of truth. The cloud evaluation used one database host with no standby, so it says nothing about database high availability.

3. The event log is never garbage collected; retention is future work.

4. Semantic deduplication is opt-in and conservative. A false negative is no worse than exact matching, but a false positive would suppress an action the user wanted, so the semantic mode must be requested per activity and every verdict is journalled.

5. All cloud measurements come from one availability zone on burstable (“flex”) instance types, over individual runs of a few minutes. They establish the shape of the behaviour on this configuration, not absolute capacity for other hardware.

1.6 Contributions

The contributions of this work are as follows.

1. A durable execution engine in which event sourcing, replay, a durable task queue, timers, signals, sagas and shard failover are implemented over PostgreSQL alone, with no message broker or coordination service.

2. Fencing at the persistence layer: every transaction that writes shard state re-reads the shard's fencing token with `SELECT ... FOR UPDATE` and commits only if the token is still the writer's. On the cloud test bed this rejected every write attempted by a stale owner after a process freeze.
3. A two-level mapping in which workflows hash to a fixed set of shards and shards are assigned to nodes by a consistent-hash ring, so that a change of membership moves ownership but no data.
4. A semantic idempotency check that compares the meaning of an agent's intended action with the actions already in the run's history and journals the verdict, keeping replay deterministic and auditable.
5. A cloud evaluation on Amazon EKS covering idle overhead, latency floor, closed- and open-loop capacity, sensitivity to workers, shards and replicas, and recovery from six classes of injected fault, together with the evaluation harness and its validity checks.

1.7 Organisation of the Report

Chapter 2 reviews related work on durable execution, partition management, ownership and fencing, sagas, and semantic similarity, and identifies the gap this work addresses. Chapter 3 states the requirements and the analysis that led to the chosen approach. Chapter 4 presents the high-level and low-level design. Chapter 5 describes the implementation and Chapter 6 the testing methodology, including the cloud test bed and the fault-injection procedure. Chapter 7 reports and discusses the results, and Chapter 8 concludes and outlines future work.

CHAPTER - 2

LITERATURE SURVEY

This chapter surveys the work on which this project builds and against which it must be positioned. The survey is organised by the six areas the design touches rather than chronologically, because the contribution of this work lies precisely at the intersection of two of them — durable execution and embedding-based similarity — that have so far been studied in isolation.

A note on venues is warranted. In distributed systems, the top conferences — VLDB, SOSP, CIDR, NeurIPS and IPDPS — are fully peer-reviewed and carry standing equal to or greater than most journals. This is the norm of the subfield rather than a substitute for journal publication. Except where a foundational result is explicitly identified as such, every work surveyed here was published between 2020 and 2026.

2.1 Durable Execution and Workflow Correctness

The idea that a long-running computation should be made resilient by recording its progress rather than by checkpointing its memory is the foundation of this project. AMBROSIA [16] framed the property as *virtual resiliency*: application code is written as though failures do not occur, and the runtime obtains resilience by logging the messages that cross the application boundary and replaying them after a crash. AMBROSIA is the closest published system to this project in intent, and it is also the sharpest point of comparison on recovery performance; its reported automatic failover in approximately 1.8 seconds with no loss is better than what this project achieves, a point returned to in Section 4.1.6 and reported plainly rather than omitted.

The DBOS line of work makes a different and complementary argument: that the right place for this machinery is inside a database. The original DBOS paper [4] proposed moving operating-system-level state into a DBMS, accepting a measured penalty of roughly forty times the cost of a conventional system call in exchange for transactional state management and queryable system state. *DBOS: Three Years Later* [**?**] reported on the practical consequences after the idea had been built out, describing a durable ex-

ecution system constructed directly on PostgreSQL. That paper is the closest academic precedent for this project's central engineering decision — persistence-layer durability with no external orchestrator — and its stated limitations are instructive: execution is not partitioned across machines, and calls to external services must still protect themselves against repetition.

R3 [5] provides the formal treatment of record-and-replay for database-backed applications, including the *retroaction* capability of replaying a recorded execution against modified code. Its central limitation is directly relevant here: external API calls cannot be replayed unless they are safe to repeat, which is the same boundary this project confronts in its crash-window analysis.

Most recently, AC/DC [**?**] addressed consistency and correctness guarantees for data-oriented workflow systems, and contributed two findings this project uses. The first is the quantitative observation already cited in Chapter 1, that error-handling logic dominates application code. The second is an experimental comparison between locking and compensation-based rollback: when a single workflow step took one second, their locking approach degraded to approximately two orders per second with 42% of orders failing, whereas the step-by-step compensation approach sustained sixteen orders per second with no failures. That result is the empirical justification for this project's decision to implement compensation rather than long-held locks, and it is referenced again in Section 4.2.9.

Netherite [19] addressed the efficient execution of serverless workflows, and is relevant to this work less for its mechanism than for its evaluation methodology: it reports scaling as a ratio rather than as an absolute throughput, which is the correct way to report a scaling result obtained on hardware that differs from the reader's.

2.2 Sharding and Partition Management

Partitioning execution across machines is a well-understood problem with one dominant published reference point. Meta's Shard Manager [6] analysed hundreds of internal sharded applications and generalised them into a single framework managing shards across approximately a million machines. Two findings matter to this project.

The first is architectural confirmation: Shard Manager also separates the mapping from application key to shard from the mapping of shard to server, exactly the two-level in-

Table 2.1: Research on durable execution and workflow correctness

Work	Venue, year	Contribution	Stated limitation
AMBROSIA [16]	PVLDB 2020	Virtual resiliency by logging messages at the application boundary	Requires the application to adopt its programming model throughout
DBOS [4]	VLDB 2022	Places operating-system state in a DBMS for transactional management	Database calls measured at roughly 40× the cost of a system call
R3 [5]	PVLDB 2023	Formal record–replay–retroaction for database-backed applications	External API calls cannot be replayed unless repeatable
Netherite [19]	PVLDB 2022	Efficient serverless workflow execution; ratio-based scaling evaluation	Reports scaling ratios without absolute baselines
DBOS 3 Years [**?**]	VLDB Journal 2025	Durable execution built directly on PostgreSQL	No partitioning across machines; external calls self-protect
AC/DC [**?**]	CIDR 2026	Correctness guarantees and rollback for workflow systems	Locking variant: 42% failures at 1 s steps; single machine only

direction adopted in Section 4.1.4, and for the same reason — it makes rebalancing an assignment change rather than a data migration.

The second is a limitation this project deliberately avoids. Shard Manager depends on ZooKeeper for coordination. At Meta's scale that is unremarkable; at the scale this project targets, a second distributed system whose only job is to decide who owns what is a substantial and avoidable cost. The observation that even a system of that sophistication still requires an external coordinator is the motivating evidence for this project's persistence-layer alternative.

Table 2.2: Production workflow orchestrators and the infrastructure each requires, from their own deployment documentation

System	Origin	Purpose	Components required
Cadence	Uber	Long-running business processes; ≈12 billion task runs per month	Cassandra or MySQL, queues, timer service, matching service, workers
Conductor	Netflix	Video encoding and delivery pipelines running for days	Database, Elasticsearch, queues, central server, workers
Airflow	Apache	Scheduled data pipelines	PostgreSQL, Redis or RabbitMQ, scheduler, Celery workers
Quartz	Enterprise Java	Scheduled jobs across many servers without duplication	Shared database with row locks
Temporal	Temporal	General-purpose durable execution	Datastore, history service, matching service, frontend
Restate	Restate	Log-native durable execution	Built-in replicated log and embedded RocksDB
durexec	**This work**	**General-purpose durable execution**	**PostgreSQL**

2.3 Ownership, Coordination and Fencing

The question of which machine is allowed to act on behalf of a partition at a given instant is the hardest correctness problem in a sharded system, because the failure mode is silent. A node that has been frozen by a garbage-collection pause or a virtual-machine

suspension has no way to detect that time has passed, and will resume in the sincere belief that it still holds a lease that has in fact expired and been granted to someone else.

The canonical treatments of this problem are Raft, which obtains agreement through an explicit consensus protocol at the cost of operating a dedicated cluster, and the *fencing token* argument, which observes that a lease is not sufficient and that the protected resource must itself reject writes carrying a stale token [9]. Temporal implements precisely this idea in its persistence layer under the name *RangeID*.

The design adopted in Section 4.1.5 follows the fencing-token argument but places the check inside the database transaction that performs the write, using `SELECT … FOR UPDATE` to serialise the two. Nothing in this mechanism is novel in isolation; what is claimed is that the combination is sufficient, that it removes the need for ZooKeeper in a system of this scale, and that its cost — the database becomes the coordination bottleneck and the availability floor — is acceptable and should be stated rather than concealed.

2.4 The Saga Pattern and Compensation

When a multi-step transaction fails partway through and its completed steps cannot be rolled back by the database, the established remedy is the saga: run a compensating action for each completed step, in reverse order. The pattern originates in Garcia-Molina and Salem's 1987 formulation [25], included here as the foundational reference despite falling outside the recency window of this survey.

Recent work has extended it in two directions relevant to this project. Daraghmi et al. [27] addressed the saga's lack of isolation — the fact that other transactions can observe intermediate states between a step and its compensation — by introducing a quota cache. This project does not solve that problem, and Section 4.2.9 states plainly that it accepts the limitation rather than addressing it. Reza and Rahman [**?**] provide a formal extension of saga and microservice patterns for resilient distributed transactions.

The most directly competing work is Robust Agent Compensation (RAC) [**?**], a log-based compensation framework built specifically for AI agents, which uses the Model Context Protocol to discover compensation pairs. RAC and this project address adjacent halves of the same problem and the distinction is worth stating precisely: **RAC undoes an agent's actions after a failure has been detected, whereas this project prevents**

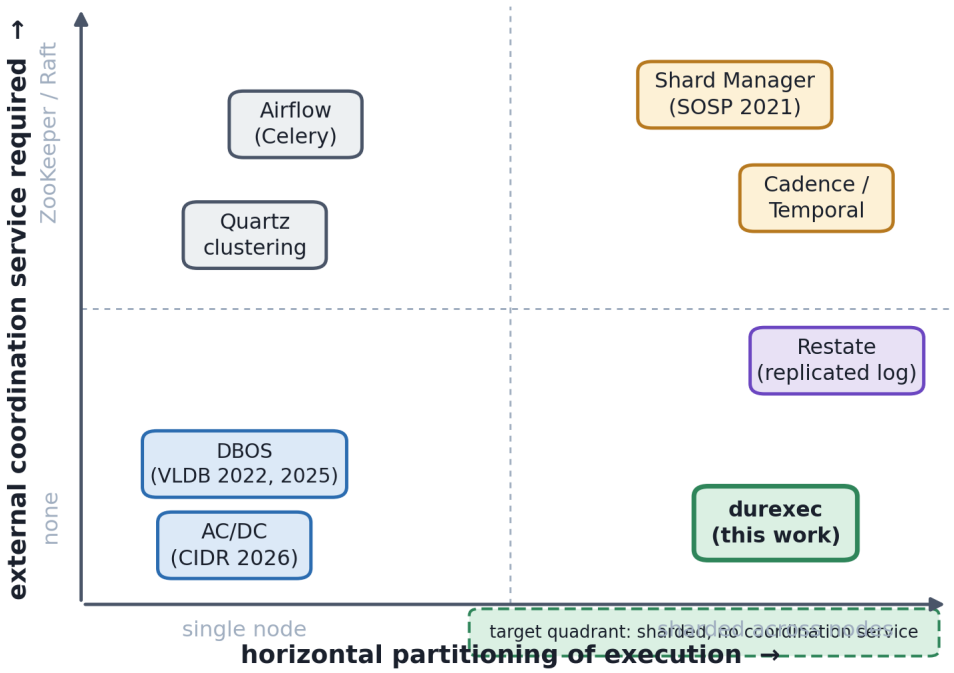


Fig. 2.1: Positioning of the surveyed systems on the two axes this project cares about. Systems in the upper half require a coordination service outside their data store; systems in the right half partition execution across machines. The lower-right quadrant — partitioned without a coordination service — is the target of this work.

a semantically duplicate action from executing in the first place. RAC, by construction, cannot observe a duplicate that succeeds, because a successful duplicate triggers no failure and therefore no compensation. That is exactly the case the mechanism in Section 4.2.10 is designed to catch.

Table 2.3: Partitioning, ownership and compensation literature

Work	Venue, year	Contribution	Relevance to this work
Shard Manager [6]	SOSP 2021	Generic shard management across $\approx 10^6$ machines	Confirms the two-level mapping; still requires ZooKeeper
Daraghmi et al. [27]	Applied Sciences 2022	Saga extension with a quota cache for isolation	Identifies the isolation gap this work accepts
Reza and Rahman [**?**]	Springer LNNS 2023	Formal saga and microservice extension	Supports the compensation-manager design
RAC [**?**]	ACM CAIS 2026	Log-based compensation for AI agents via MCP	Closest competing work; acts after failure, not before

2.5 Failure Modes of LLM Agent Systems

The premise of this project's research contribution is that agent-driven workflows repeat actions in ways that exact-match deduplication cannot catch. That premise requires evidence rather than assertion, and the evidence exists.

Why Do Multi-Agent LLM Systems Fail? [11] analysed more than 1,600 annotated failure traces across seven agent frameworks and produced a taxonomy in which *step repetition* appears as a distinct, measured failure category. *Where LLM Agents Fail and How They Can Learn From Failures* [**?**] introduces a modular failure taxonomy spanning planning, memory and action modules together with an annotated trajectory dataset. The Microsoft AI Red Team's *Taxonomy of Failure Modes in Agentic AI Systems* [12] documents the same class from an industrial security perspective.

A useful contrast is provided by work that resolves the same tension in the opposite direction. Compiled AI [**?**] obtains determinism in LLM-based workflow automation by removing the model from the execution loop entirely, generating deterministic code ahead of time. That is a coherent position, and it is the opposite of the one taken here:

this project preserves the agent's runtime flexibility and addresses the resulting non-determinism at the point where it causes harm, namely the duplicated side effect.

Table 2.4: Literature establishing that agent step repetition is a real, measured failure mode

Work	Venue, year	Finding relevant to this project
Cemri et al. [11]	NeurIPS 2025	1,600+ annotated failure traces across seven frameworks; step repetition is an identified category
Zhu et al. [**?**]	arXiv 2025	Modular failure taxonomy across planning, memory and action, with an annotated trajectory dataset
Kumar et al. [12]	Microsoft AI Red Team, 2025	Industrial taxonomy of agentic failure modes, independently documenting the same class
Compiled AI [**?**]	arXiv 2026	Achieves determinism by removing the model from the loop — the opposite trade-off to the one made here

2.6 Semantic Caching and Embedding-Based Similarity

This is the body of work closest to the technique this project applies, and the one against which the novelty claim must be made most carefully.

GPTCache [30] established embedding-similarity matching as a recognised technique for large-language-model applications: if a new prompt is sufficiently similar to a prompt already seen, return the cached response instead of recomputing it. MeanCache [31] extended the idea to a user-centric, federated setting. Biton and Friedman [**?**] frame the exact-match versus semantic-match distinction in terms almost identical to the `Exact` and `Semantic` idempotency modes defined in Section 4.2.10, and Gill et al. [**?**] examine the practical question of which embedding model and threshold to use.

Two recent results bear directly on the design and on the honest reporting of its weaknesses. *Closing the Calibration Gap in Semantic Caching* [**?**] argues that a single similarity threshold forces a binary decision at precisely the point where the model is least confident, and motivates the two-threshold design with an escalation band adopted in Section 4.2.10. vCache [**?**], evaluated across roughly 45,000 examples, reports that a single fixed similarity cut-off cannot reliably separate true duplicates from genuinely

distinct requests, with the problematic thresholds landing in the same 0.84–0.85 region this project uses. That result independently corroborates the negative finding this project reports about its own semantic comparator, and it is cited for that purpose rather than being omitted.

Table 2.5: Semantic caching and embedding-similarity literature

Work	Venue, year	Contribution and its bearing on this design
GPTCache [30]	ACL NLP-OSS 2023	Establishes embedding similarity for response caching; the technique this work repurposes
MeanCache [31]	IEEE IPDPS 2025	User-centric federated semantic cache for LLM web services
Gill et al. [**?**]	arXiv 2025	Domain-specific embeddings and synthetic data for threshold selection
Biton and Friedman [**?**]	arXiv 2026	Frames exact versus semantic matching as this work frames its two idempotency modes
Calibration gap [**?**]	arXiv 2026	Motivates the two-threshold design with an escalation band
vCache [**?**]	ICLR 2026	Shows a single cut-off cannot separate duplicates from distinct requests at ≈45,000 examples; corroborates this project's negative result

2.7 Comparative Synthesis

Five problems recur across the surveyed work, and the design presented in Chapter 4 is in each case a direct response to one of them.

First, systems accumulate components. Airflow, Conductor and Cadence each require a database, a queue, a scheduler and a worker fleet; Shard Manager adds ZooKeeper. Every component is an independent failure domain.

Second, error handling dominates application code. AC/DC's measured ratio of ninety to ten [**?**] shows that the cost is paid by every application author individually unless the engine absorbs it generically.

Third, long-held locks are not a viable rollback mechanism. AC/DC's own comparison — 42% failure under locking against no failures under compensation at one-second steps — settles this experimentally.

Fourth, exactly-once execution across a process boundary is not achievable. DBOS [**?**], R3 [5] and AMBROSIA [16] each arrive at the same conclusion from different directions: the system must ultimately rely on the downstream resource to recognise a repeat. The honest engineering response is to narrow the window and supply a stable key, which is what Section 4.2.8 does.

Fifth, and newest, agent re-planning defeats byte-level duplicate detection. The evidence is in [11**?** , 12]; the mechanism that would detect it exists in [30**?** , 31]; and no published work connects the two.

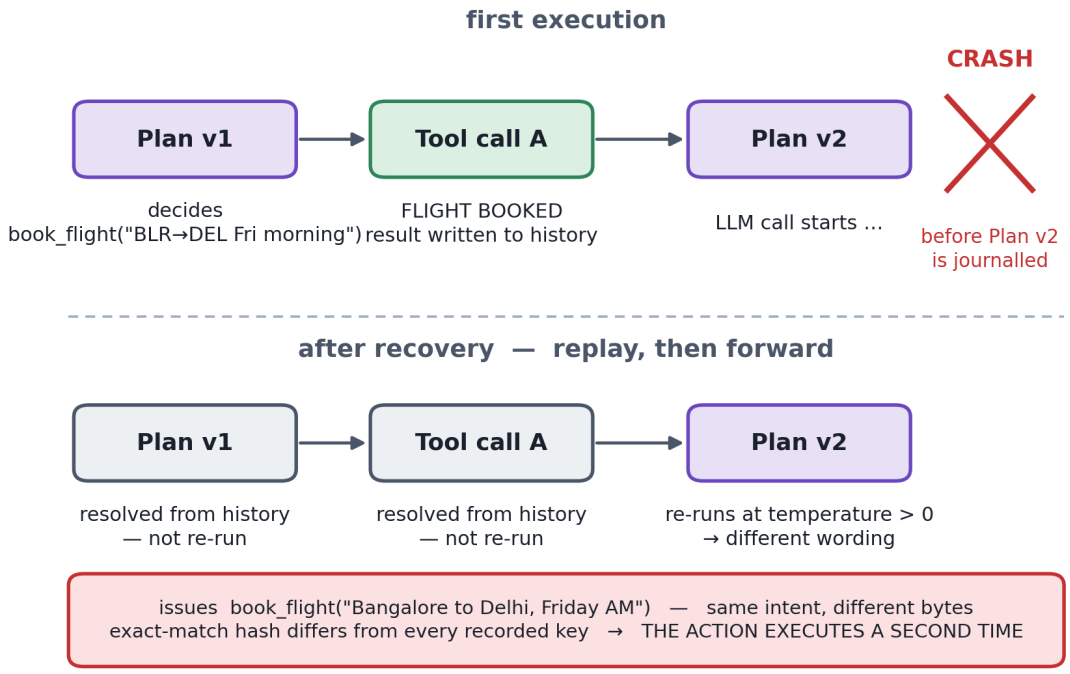


Fig. 2.2: The crash-induced, semantically duplicate side effect that exact-match idempotency cannot detect. Completed steps resolve from history and are not re-run, but a planning step that re-executes at a non-zero sampling temperature re-issues the same intent in different words, producing a hash that matches no recorded key.

2.8 Identified Research Gap

The gap this project addresses can be stated in a single sentence.

> Every work in the semantic-caching literature applies embedding similarity to decide whether to skip *recomputing an answer*; none applies it to

decide whether to skip *re-executing a side-effecting action* inside a durable execution engine.

The distinction is not cosmetic, because the consequence structure of a wrong decision is entirely different. A semantic cache that returns a wrong hit costs the user an inaccurate answer and costs the system nothing; the error is recoverable and usually visible. A duplicate-suppression mechanism that returns a wrong hit silently prevents a real-world action from occurring — a flight the user wanted is never booked, and nothing reports an error. Three consequences follow, and they shape the design in Section 4.2.10:

1. The threshold must be calibrated for a different and far more conservative error cost than a cache's, which is why the two-threshold design of [**?**] is adopted and why the mechanism is opt-in per activity rather than global.
2. The verdict must be *deterministic under replay*. A cache may recompute its decision freely; a durable execution engine may not, because a verdict that differed between the original execution and a replay would break the invariant on which the entire engine rests. This requirement has no counterpart in the caching literature and is the technically interesting part of the design.
3. The verdict must be *auditable after the fact*. When a mechanism is empowered to suppress real-world actions, the ability to answer "why did this not happen?" with a stored row rather than a guess is not a convenience; it is what makes the feature deployable at all.

Table 2.6 summarises how this work is positioned against the closest prior art in each area.

Table 2.6: Positioning of this work against the closest prior art

Area	Closest prior art	What it does	What this work does
Durable execution	DBOS [**?**]	Postgres-backed, single node	Same substrate, partitioned across nodes
Partitioning	Shard Manager [6]	Two-level mapping with ZooKeeper	Two-level mapping, no coordinator
Ownership	Fencing tokens [9]	Resource rejects stale tokens	Check moved inside the write transaction
Compensation	RAC [**?**]	Undoes agent actions after failure	Prevents the duplicate before it runs
Deduplication	GPTCache [30]	Skips recomputing an answer	Skips re-executing a side effect
Calibration	vCache [**?**]	Shows one cut-off is insufficient	Reproduced independently and reported

CHAPTER - 3

SYSTEM REQUIREMENTS AND ANALYSIS

This chapter derives the requirements the engine must satisfy, specifies the software and hardware environment in which it is built and measured, and records the assumptions and constraints under which the design in Chapter 4 was made. Requirements are stated so that each is testable: a requirement that cannot be checked by an automated assertion against a running cluster has been reformulated until it can be.

3.1 Functional Requirements

The functional requirements fall into six groups, corresponding to the six mechanisms the engine implements. Tables 3.1 and 3.2 list them with the identifier used throughout the remainder of the report.

Two of these deserve comment because they are unusual as requirements.

FR-3 requires the engine to *fail* rather than to recover. When replay produces a different sequence of decisions from the original execution, the engine cannot know which version of the code told the truth, and any attempt to reconcile the two would silently corrupt the history. Refusing to continue is the only safe behaviour, and detecting the condition at all is a non-trivial design obligation discharged in Section 4.2.3.

FR-8 is the central safety requirement of the whole system and is stated in the negative deliberately. It is not enough that ownership transfers; it must additionally be impossible for the previous owner to act, even though that node has no way of knowing it has been displaced. Section 4.1.5 gives the mechanism.

3.2 Non-Functional Requirements

NFR-6 is a constraint rather than a goal, and it is the one that most shapes Chapter 4. Several otherwise attractive designs — a Redis-backed task queue, a Kafka-backed event log, a ZooKeeper-based ownership service — are excluded by it. The justification is the

Table 3.1: Functional requirements

ID	Requirement	Verified by
FR-1	Every workflow state transition shall be persisted as an immutable event before it is acted upon	Assert no `UPDATE` or `DELETE` is ever issued against the event log
FR-2	A workflow interrupted at any point shall resume from its recorded history without re-executing completed activities	Kill a node mid-workflow; compare completed-activity counts before and after
FR-3	The engine shall detect and refuse to continue a workflow whose code takes a different branch on replay	Inject a deliberate non-determinism; assert the workflow fails loudly
FR-4	A failed activity shall be retried according to a configurable policy with exponential backoff	An activity failing three times then succeeding shall produce exactly one success event
FR-5	The engine shall supply each activity with a deduplication key that is stable across crashes and replays	Assert the key observed by the activity is identical on every attempt
FR-6	Workflow execution shall be partitioned across a fixed number of shards, and each workflow shall remain in one shard for its lifetime	Assert shard assignment is unchanged across the full life of 1,000 workflows
FR-7	Shard ownership shall transfer automatically when the owning node fails	Kill the owner; assert another node acquires the lease and resumes work
FR-8	A node that has lost ownership shall be unable to commit any change to that shard	Freeze a node past its lease expiry; assert its transaction rolls back
FR-9	Adding or removing a node shall move ownership of a minority of shards and migrate no workflow data	Add a fourth node to a three-node cluster; count reassigned shards

Table 3.2: Functional requirements (continued)

ID	Requirement	Verified by
FR-10	A workflow shall be able to sleep for an arbitrary duration without holding a thread, connection or memory	Start a workflow with a two-minute sleep; restart the whole cluster; assert it wakes on schedule
FR-11	An external signal shall be durable before any workflow code observes it	Send a signal and kill the cluster immediately; assert the signal survives
FR-12	A multi-step transaction failing partway shall run compensations for completed steps in reverse order	Force a late step to fail; assert compensation events appear in LIFO order
FR-13	A compensation that exhausts its retries shall leave the workflow in a distinct terminal state requiring human intervention	Assert status `COMPENSATING_FAILED` is reachable and surfaced
FR-14	For activities configured in semantic mode, a semantically equivalent repeated action shall be suppressed before it executes	Count real side effects in an external ledger across an induced agent re-plan
FR-15	Every deduplication verdict shall be journalled with its similarity score, threshold and matched predecessor	Assert the verdict event is present and queryable through the API
FR-16	The engine shall expose workflow start, query, history retrieval, signalling and cancellation over HTTP	Contract tests against each documented endpoint
FR-17	The engine shall export metrics suitable for a time-series database	Scrape `/metrics` and assert the expected series exist
FR-18	The engine shall provide a fault-injection facility that crashes a node at a named point in the execution path	Assert each named crash point is reachable and produces the documented state

Table 3.3: Non-functional requirements

ID	Attribute	Requirement
NFR-1	Correctness	Zero committed work shall be lost under induced node failure. This is an absolute requirement, not a target.
NFR-2	Correctness	Zero activities shall execute twice as a result of a failover, excluding the irreducible crash window of Section 4.2.8.
NFR-3	Availability	A shard shall be unavailable for at most the lease duration following the loss of its owner. With the default ten-second lease this bounds unavailability to ten seconds for $1/N$ of workflows.
NFR-4	Scalability	Throughput shall increase materially when shard count is doubled; the scaling ratio, not the absolute rate, is the reported figure.
NFR-5	Latency	Reconstructing a workflow's state from a two-hundred-event history shall complete in under fifty milliseconds at the 99th percentile.
NFR-6	Operability	The system shall require exactly one infrastructure dependency. No message broker, coordination service or search index may be introduced.
NFR-7	Portability	A node shall be a single statically linked binary, deployable as a container image without a runtime interpreter.
NFR-8	Observability	Every failover, retry, compensation and deduplication verdict shall be visible as a metric or as a queryable event.
NFR-9	Resource footprint	The complete demonstration cluster shall run within the memory of a single developer workstation.
NFR-10	Auditability	Any suppressed side effect shall be explainable from stored data alone, without re-running any model.
NFR-11	Security	No component intended for testing shall be reachable from outside the host, and no credential shall appear in source control.

argument of Section 1.2.1: for the deployment scale this project targets, each additional component costs more in operational risk than it returns in capability.

NFR-9 deserves a candid note. The demonstration cluster was sized to fit a single workstation because that is the hardware available, and this constrains what the evaluation can claim. It does not constrain correctness results — whether work was lost or duplicated is independent of how fast the machine is — but it does mean that absolute throughput figures from this project are not comparable with figures from systems evaluated in data centres. This is restated in Section 3.5 as an explicit constraint on the evaluation.

3.3 Software Requirements

Three choices in Table 3.4 were contested during design and are justified here rather than left implicit.

Go rather than Java or Rust. The workflow dispatcher of Section 4.2.4 requires a construct that can suspend a function mid-execution and resume it later under the engine's control rather than the operating system's. Go's goroutines and unbuffered channels express this handshake in a few lines. Java would require either threads, which are too heavy for one per in-flight workflow, or a bytecode-level continuation library. Rust's `async` machinery would work but introduces lifetime complexity disproportionate to the benefit at this scale.

PostgreSQL rather than a key-value store. The engine's correctness rests on committing a workflow decision and all of its consequences — new events, new task rows, new timer rows, an updated execution pointer — in a single atomic transaction. A key-value store without multi-key transactions would require the engine to implement that atomicity itself, which is precisely the complexity the design is trying to remove.

A relational table rather than a dedicated queue. `FOR UPDATE SKIP LOCKED` allows many workers to poll the same table concurrently without serialising behind one another, which is the property that makes an external broker unnecessary. Section 4.2.6 gives the query and explains why the clause is load-bearing rather than an optimisation.

Table 3.4: Software requirements

Component	Version	Role and justification
Go	1.25.10	Implementation language. Chosen for goroutines, which make the cooperative scheduler of Section 4.2.4 expressible, and for static linking, which satisfies NFR-7.
PostgreSQL	16	The sole infrastructure dependency. Supplies the event log, the task queue, the timer store and the ownership record. Chosen for `SKIP LOCKED`, `FOR UPDATE` and streaming replication.
pgvector extension	bundled in the container image	Stores 384-dimensional embeddings for the semantic idempotency comparator.
jackc/pgx	v5.10.0	PostgreSQL driver and connection pool. Chosen over `database/sql` for native protocol support and explicit pool statistics.
Docker Engine and Compose	current stable	Reproducible multi-node deployment on one host.
Prometheus	v2.55.1	Metrics collection, satisfying NFR-8.
Grafana	11.2.0	Dashboards over the collected metrics.
client_golang	v1.24.1	Metric instrumentation inside the engine binary.
`all-MiniLM-L6-v2`	—	The sentence-embedding model behind the semantic comparator. Chosen for its small size and 384-dimensional output, which keeps the vector column narrow.

3.4 Hardware Requirements

This is a software project; the hardware requirement is that of the development and measurement environment rather than of a manufactured artefact. Table 3.5 records it, and Fig. 3.1 shows the topology that runs on it.

Table 3.5: Hardware requirements of the development and measurement environment

Resource	Minimum	Rationale
Processor	4 cores, x86-64	Three engine containers plus two PostgreSQL instances plus the observability stack must run concurrently without the measurement competing with the system being measured.
Memory	8 GB usable	The base stack occupies approximately 2.3 GB; adding the embedding model raises this to approximately 3.2 GB.
Storage	20 GB free	Container images, two PostgreSQL data directories, and an event log that has no garbage collection.
Network	Loopback only	The entire cluster runs on one host; no component is exposed beyond it, satisfying NFR-11.
Accelerator	None required	The embedding model runs acceptably on CPU for the candidate-set sizes involved, which are tens of vectors rather than millions.

3.5 Assumptions and Constraints

Assumption A-3 is worth expanding, because clock handling is where lease-based systems most often go wrong. The engine never compares a timestamp generated on one node against a timestamp generated on another. Lease expiry is expressed as `expires_at < now()` and evaluated inside the database, so all comparisons occur against a single clock. A node whose own clock is badly wrong will renew its lease too eagerly or too

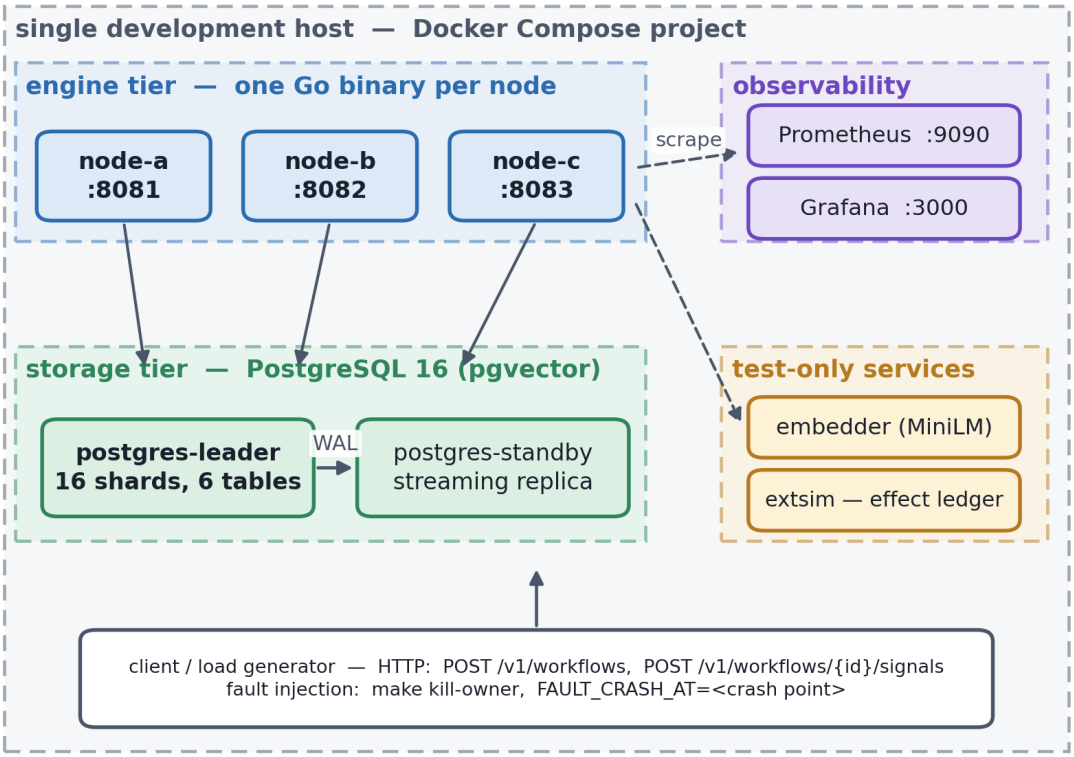


Fig. 3.1: The container topology against which the requirements of this chapter are sized, and on which every measurement reported by this project was taken. Three engine nodes share one PostgreSQL leader with a streaming standby; the embedder and the external-effect simulator are test-only services and are never exposed beyond the host.

Table 3.6: Assumptions and constraints under which the design was made

ID	Type	Statement
A-1	Assumption	PostgreSQL provides linearizable reads and writes for a single row under `FOR UPDATE`. The entire ownership-safety argument depends on this and on nothing else.
A-2	Assumption	Workflow authors write deterministic orchestration code. The engine detects violations (FR-3) but cannot repair them.
A-3	Assumption	Clocks on different nodes may disagree, but each node's view of elapsed time is monotonic. Lease expiry is evaluated by the database, not by the nodes, specifically so that cross-node clock skew cannot affect correctness.
A-4	Assumption	Downstream services honour an idempotency key when one is supplied. The irreducible crash window of Section 4.2.8 can only be closed by the resource itself.
C-1	Constraint	Exactly one infrastructure dependency is permitted (NFR-6).
C-2	Constraint	Evaluation hardware is a single workstation. Absolute throughput is therefore not comparable with published data-centre figures; scaling ratios and correctness counts are.
C-3	Constraint	Ownership uses lease renewal backed by a linearizable database rather than a full consensus protocol. This is a deliberate scope decision recorded in Section 1.5.
C-4	Constraint	The semantic comparator must not be consulted during replay, because recovery may not depend on the availability of an external model. This constraint dictates the design of Section 4.2.10 almost entirely.
C-5	Constraint	No credential may be committed to source control, and no test-only service may be reachable from outside the host.

lazily, but it cannot cause two nodes to believe simultaneously that they own the same shard, because the decision is made in one place.

3.6 Analysis of the Chosen Approach

The requirements above admit several architectures. Table 3.7 records the principal alternatives considered and the reason each was rejected, so that the design presented in Chapter 4 is understood as a choice among options rather than as the only possibility.

The resulting position can be summarised in one sentence: **the engine keeps the single-dependency simplicity of the DBOS line of work while adding the horizontal partitioning that line does not attempt, and it obtains the ownership safety that normally justifies a coordination service from a fencing token re-checked inside each write transaction.** Chapter 4 develops that position into a complete design.

Table 3.7: Architectural alternatives considered and reasons for rejection

Alternative	Attraction	Reason rejected
Build on Temporal or Restate	Mature, production-proven, saves months	The project *is* the implementation; and the semantic idempotency contribution requires modifying the engine's decision path, which cannot be done from outside as a user of the platform
Kafka as the event log	Purpose-built append-only log with replication	Violates NFR-6; and a workflow decision plus its task and timer rows could no longer be committed atomically with the events
Redis or RabbitMQ task queue	Lower polling latency than a database table	Violates NFR-6; `SKIP LOCKED` already provides concurrent non-blocking dequeue within the existing dependency
ZooKeeper or etcd for ownership	The textbook answer, used by Shard Manager [6]	Violates NFR-6; a fencing token re-checked inside the write transaction provides the same safety property (Section 4.1.5)
Full Raft for ownership	True consensus, with no reliance on a single database	Disproportionate to the scale targeted; recorded as future work in Section 1.5
One database per shard	Removes the shared-database bottleneck	Every key already carries the shard identifier, so this is a deployment change; deferred rather than rejected
Single-node design, as in DBOS [**?**]	Far simpler; no ownership problem at all	Objective 4 requires horizontal partitioning, which is precisely the aspect DBOS does not address
Lock-based rollback instead of sagas	Stronger isolation between concurrent workflows	Measured by AC/DC [**?**] to fail 42% of orders at one-second steps; compensation is the empirically supported choice

CHAPTER - 4

SYSTEM DESIGN

This chapter presents the design of the engine in two parts. Section 4.1 establishes the principle from which the architecture is derived, describes the resulting structure, and justifies the decisions that the structure encodes. Section 4.2 descends to the level of individual tables, queries, transactions and goroutine handshakes, and includes the failure analysis that determines what the engine can and cannot promise.

The chapter is written so that each mechanism is introduced by the problem it solves. Where a design decision was contested, the rejected alternative is stated along with the reason for rejecting it, because a design presented without its alternatives cannot be assessed.

4.1 High Level Design

4.1.1 The Governing Design Principle

The entire architecture is derived from one invariant, stated here and referred to throughout the remainder of the chapter as the governing principle:

> **Workflow code is a pure function of its event history.** Given the same list of events, executing the workflow function produces the same sequence of decisions, every time, on any machine.

This is not a stylistic preference. It is the property that purchases every capability the engine claims, and the relationship is worth making explicit.

Crash recovery requires no snapshot of memory, threads or stack frames. The engine does not attempt to restore a process to the state it was in; it re-executes the function against the saved history, and the stack rebuilds itself as a side effect of that execution. A mechanism that saved and restored memory would have to serialise closures, open connections and goroutine stacks, none of which is portable between machines or across code versions.

Migration between machines is free. A workflow can resume on a completely different node because nothing machine-local was ever part of its state. There is no affinity to re-establish and no cache to warm. This is what makes shard failover a matter of ownership rather than of data movement.

Debuggability is structural rather than added. The history *is* the execution trace; there is no separate log to correlate against it, and no possibility of the two disagreeing.

Everything else in this chapter is either machinery that keeps this principle true, or machinery that keeps the history durable and available.

Two consequences constrain the programming model and are enforced rather than merely documented. First, orchestration code may perform no input or output of its own, because any value read from the environment — the clock, a random source, a file, a network response — would differ between the original execution and the replay, and would break determinism. Second, any unavoidable non-determinism must be *journalled*: executed once, its result written to the log, and thereafter read from the log rather than recomputed. Section 4.2.10 applies exactly this technique to the semantic deduplication verdict, which is what makes that contribution defensible rather than ad hoc.

4.1.2 Architectural Overview

The system is organised into five tiers, shown in Fig. 4.1 and summarised in Table 4.1. The tiers are logical rather than physical: every node runs the whole stack above the persistence tier in a single operating-system process.

Three properties of this structure are deliberate and are worth stating separately.

The gateway holds no state. It hashes a workflow identifier, looks up the owning node and forwards. If its view of ownership is stale it will forward to a node that no longer owns the shard; that node replies with a `NotOwner` response naming the current owner, and the gateway refreshes and retries. Routing is therefore allowed to be briefly wrong. Correctness is enforced at the point of writing, not at the point of routing — which is the difference between a design that requires global agreement on a routing table and one that does not.

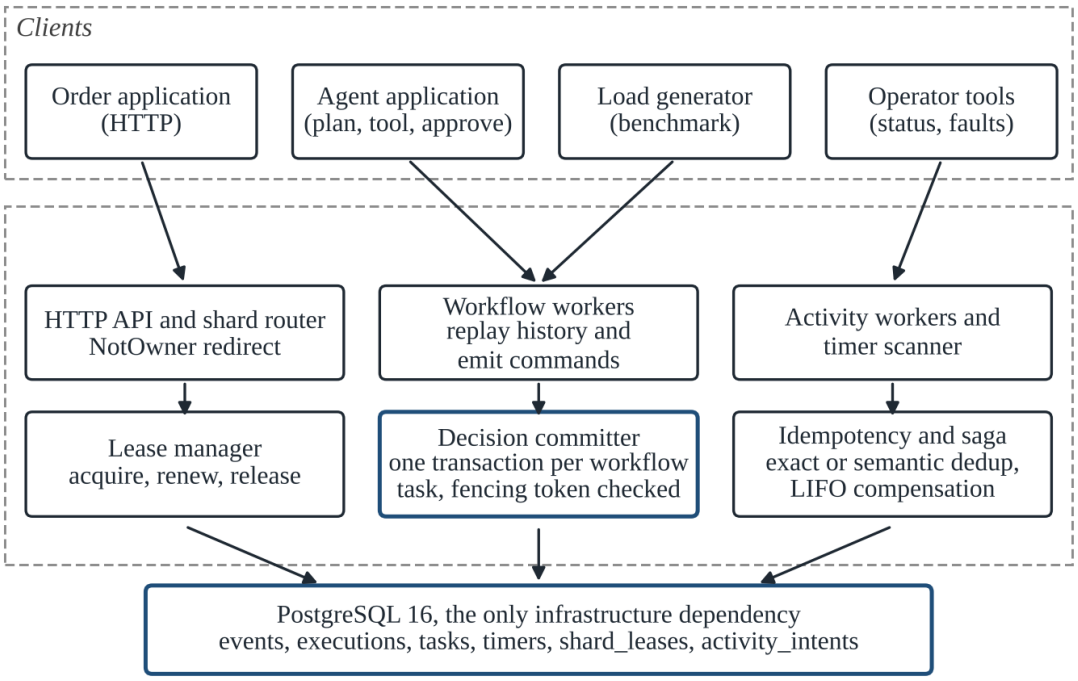


Fig. 4.1: The five architectural tiers of the engine. Every node runs the client-facing gateway, the execution tier and the control tier in one process; only the persistence tier is shared. The control tier is where every mutating path is gated by a fencing-token check, which is what makes partitioned execution safe without a coordination service.

Table 4.1: Architectural tiers, their responsibilities and their state

Tier	Responsibility	State held
Client	Starts workflows, sends signals, observes progress, injects faults during evaluation	None
Gateway	Routes a request to the node that owns the target shard; redirects when its routing view is stale	None. Deliberately stateless and replicated on every node
Execution	Replays history, decides what happens next, performs side effects, fires timers, runs compensations	Only transient per-task state, discarded after every commit
Control	Acquires and renews shard leases; verifies the fencing token inside every mutating transaction; evaluates idempotency	The fencing token currently held, in memory, re-verified on every write
Persistence	Stores events, executions, tasks, timers, leases and activity intents; replicates to a standby	All durable state in the system, without exception

The execution tier keeps nothing between tasks. When a workflow task completes, the in-memory workflow state is discarded. Correctness never depends on retaining it. Caching the workflow between tasks — what Temporal calls *sticky execution* — is a pure latency optimisation that can be added and removed without any change in behaviour.

The persistence tier is the only shared component, and therefore the only coordination point. This is the architecture's central trade-off, and it is stated plainly in Section 4.1.7.

4.1.3 The Separation of Workflow and Activity

The programming model distinguishes two kinds of code, and the engine's correctness depends on the distinction being maintained absolutely.

The practical rule is that the workflow never does I/O and the activity never makes orchestration decisions. Blurring the line does not produce an immediate error; it produces intermittent non-determinism failures that appear long afterwards and are extremely difficult to attribute. The engine therefore makes the separation structural: the context ob-

Table 4.2: The contract between workflow code and activity code

	Workflow	**Activity**	**Enforced by**
Decides	What happens next	Nothing	Review and the determinism detector
Performs	No input or output whatsoever	All external side effects	Absence of I/O in the context object
Determinism	Mandatory	Not required	Non-determinism error at replay (FR-3)
Executions	Once per recovery, many times in total	At-least-once, ideally once	The replay mechanism
Failure	Fails the workflow	Retried under policy	Retry policy on the activity

ject handed to workflow code exposes no facility for performing I/O, so the ordinary way of writing the code is also the correct way.

4.1.4 The Two-Level Mapping: Workflows, Shards and Nodes

Partitioning is expressed as two separate mappings, and keeping them separate is the single most consequential decision in the architecture:

$$\texttt{workflow_id} \xrightarrow[N \text{ fixed at deploy}]{\text{hash}} \texttt{shard_id} \xrightarrow[\approx 150 \text{ virtual nodes}]{\text{consistent hash ring}} \texttt{node}$$

The first mapping is $\texttt{shard_id} = \text{fnv32}(\texttt{workflow_id}) \bmod N$, with N fixed at deployment time and defaulting to sixteen. A workflow never changes shard for the whole of its life. That permanence is what makes its event log a stable, single-writer append stream, which in turn is what allows the primary key alone to serve as the concurrency control (Section 4.2.1).

The second mapping assigns shards to nodes through a consistent hash ring with approximately one hundred and fifty virtual nodes per physical node. Adding or removing a node moves a minority of shards and leaves every other shard untouched.

Fig. 4.2 shows why the indirection matters. Had workflow identifiers been hashed directly to nodes, adding a node would move individual *workflows* between logs while

they were still running, which is a data migration performed under load on live state. With the fixed intermediate mapping, adding a node moves only the *right to act on* a shard. No event is copied, no workflow is rewritten, and the operation reduces to a lease changing hands.

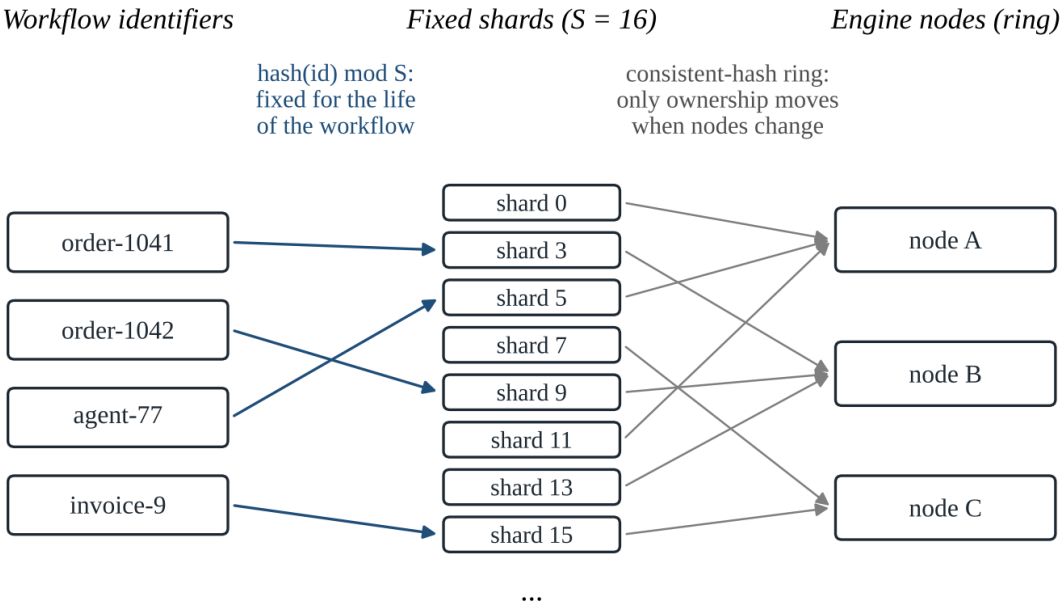


Fig. 4.2: The two-level mapping. The first mapping is fixed for the lifetime of a workflow; only the second changes when the cluster is resized. This is what reduces rebalancing from a data migration to an ownership handoff.

4.1.5 Ownership, Leases and the Fencing Invariant

This section contains the central safety argument of the design.

Ownership of a shard is recorded in a single row with four columns: the shard identifier, the owning node, a monotonically increasing fencing token, and an expiry timestamp. A node acquires a shard with the conditional update shown in Listing 4.1.

Listing 4.1: Acquiring a shard lease. Zero rows returned means another node holds a live lease.

```
UPDATE shard_leases
SET    owner_node    = $me,
       fencing_token = fencing_token + 1,
       expires_at    = now() + interval '10 seconds'
WHERE  shard_id = $shard
```

Table 4.3: Partitioning parameters and the reasoning behind each default

Parameter	Default	Reasoning
Shard count N	16	Must exceed the largest anticipated node count so that every node can own at least one shard, but must stay small enough that per-shard goroutine overhead is negligible. Fixed at deployment because changing it would re-map every workflow.
Virtual nodes per node	≈150	Enough points on the ring for the distribution across physical nodes to be near-even; a smaller number produces visible imbalance.
Hash function	FNV-1a, 32-bit	Fast, dependency-free and adequate for uniformity. Cryptographic strength is not a requirement, because the input is not adversarial.

```
  AND  (owner_node = $me OR expires_at < now())
RETURNING fencing_token;
```

Zero rows returned means someone else holds a live lease. One row returned means the caller owns the shard and now holds token T. The token increments on every change of ownership and never decreases. Renewal, performed every three seconds against a ten-second expiry, extends `expires_at` while requiring both the owner name and the token to match; a renewal returning zero rows tells the node it has been displaced.

The lease is not what makes this safe. This is the point most often missed, and it is where naive lease-based designs fail. Consider the following sequence:

1. Node A owns shard 2 with token 7. It polls a task and begins work.
2. Node A's process freezes — a garbage-collection pause, a disk stall, a virtual-machine suspension. It has no way of knowing that time is passing.
3. Ten seconds elapse. The lease expires. Node B acquires it and receives token 8.
4. Node B replays the workflow and begins executing it.
5. Node A wakes up mid-transaction, still sincerely believing it holds the lease, and attempts to commit.

No amount of care in the lease protocol prevents step five, because node A cannot detect that it was frozen. What prevents the damage is that every mutating transaction re-verifies the token *inside the same transaction as its writes*, as shown in Listing 4.2.

Listing 4.2: The fencing check. Every mutating transaction performs this before any write, and `FOR UPDATE` serialises the check against the write.

```
tx, _ := db.Begin()
var tok int64
tx.QueryRow(`SELECT fencing_token FROM shard_leases
             WHERE shard_id = $1 AND owner_node = $2
               AND expires_at > now()
             FOR UPDATE`, shard, me).Scan(&tok)
if tok != myToken {
    tx.Rollback()
    return ErrLostOwnership
}
// ... all event appends, task inserts, timer inserts ...
tx.Commit()
```

Node A's transaction reads token 8, compares it against the 7 it believes it holds, and rolls back. It commits nothing — not one event, not one task row. The `FOR UPDATE` clause is what makes this airtight: it serialises the ownership check against the write, so no window exists between "check" and "write" during which ownership could change underneath the writer. Fig. 4.3 shows the two outcomes side by side.

The resulting invariant, which satisfies FR-8, is:

> **A node that has lost ownership cannot mutate shard state, regardless of what it believes.**

This is the fencing-token argument [9] implemented in a table rather than in ZooKeeper, and it is the same technique Temporal applies under the name RangeID. Two costs must be stated honestly rather than discovered by a reader.

There is an availability gap. Between the old owner's death and the new owner's acquisition, the shard is unavailable for up to one lease duration. At the default ten-second expiry this bounds unavailability at ten seconds for $1/N$ of workflows, which is NFR-3. Shortening the lease speeds failover and increases the risk of spuriously fencing

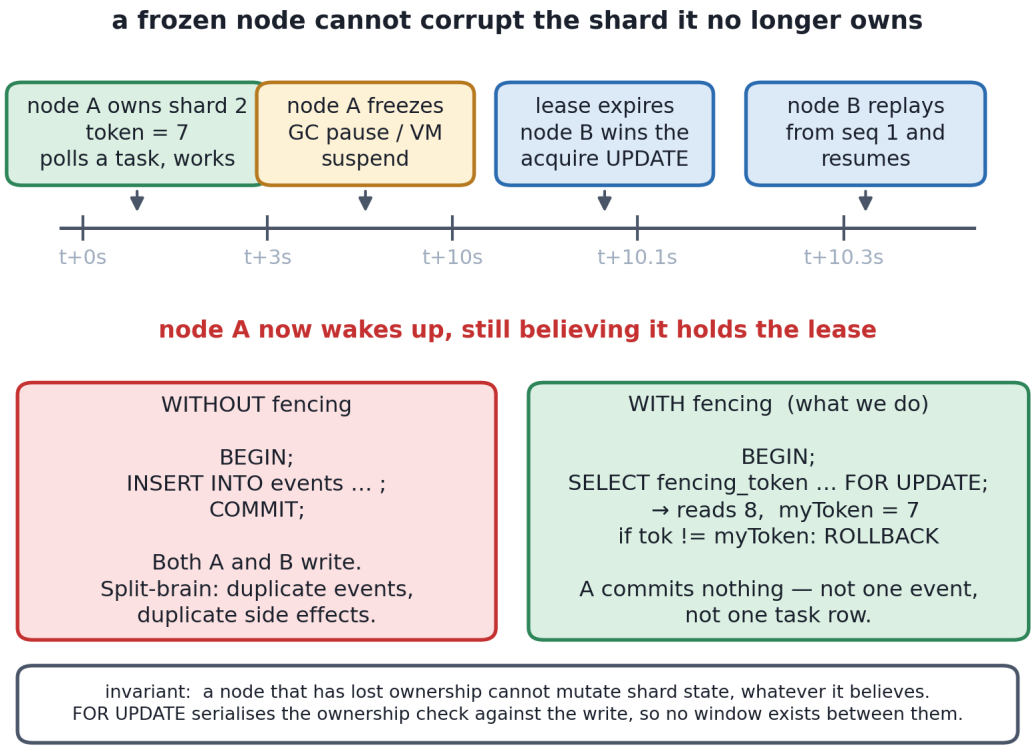


Fig. 4.3: Why the lease alone is insufficient and the in-transaction token check is what provides safety. Without fencing, a node that was frozen past its lease expiry commits alongside the new owner and produces split-brain. With fencing, it commits nothing.

a healthy but momentarily slow node; this is a genuine tunable trade-off, and the chosen value is a default rather than a discovery.

This is not consensus. The design relies on PostgreSQL as a single linearizable source of truth for the ownership record. That is a legitimate position — it is the DBOS argument [**?**] — but it must be stated plainly: the coordination problem has been moved into the database, not eliminated, and the database is consequently both the coordination bottleneck and the availability floor of the whole system.

4.1.6 Replication and Failover

Durability beyond a single machine is obtained from PostgreSQL streaming replication: a standby receives the write-ahead log from the leader and can be promoted if the leader is lost.

The choice between asynchronous and synchronous commit is a genuine trade-off rather than a setting with a right answer. Asynchronous replication (`synchronous_commit = off`) is faster but has a loss window: a leader that dies holding unreplicated commits loses them. Synchronous replication (`synchronous_commit = remote_apply`) has no loss window, but adds a network round trip to every commit and stalls entirely if the standby is unavailable. The engine defaults to synchronous commit, treating the throughput cost as a measurable result rather than as a hidden caveat.

The failover sequence following the loss of a shard-owning node is as follows, with the default timings:

1. $t + 0.0\,\text{s}$ — the node is killed. In-flight task locks are already ticking down.
2. $t + 3.0\,\text{s}$ — the dead node misses its first lease renewal.
3. $t + 10.0\,\text{s}$ — the lease expires.
4. $t{+}10.1\,\text{s}$ — a surviving node wins the acquire statement and receives the next fencing token.
5. $t + 10.3\,\text{s}$ — that node polls a workflow task, replays the history from the first event, and resumes.
6. $t + 40.0\,\text{s}$ — any activity task locks held by the dead node expire and those tasks are redelivered.

Nothing is lost, because nothing was ever held only in the dead node's memory. Some work may be *repeated*, specifically activities caught in the window analysed in Section 4.2.8, which is why stable deduplication keys exist. Recovery time is dominated by the lease expiry and should be reported as a distribution over many induced failures rather than as a single observation.

Two limitations are recorded here rather than left for a reader to find. First, **database failover in this design is slower than published work and is not automatic**: AMBROSIA [16] reports automatic failover in approximately 1.8 seconds with no loss, whereas promotion of the standby in this system is operator-initiated and takes substantially longer. Second, shards share one database instance, so the loss of that instance is a cluster-wide event rather than a per-shard one. Both are consequences of the single-dependency constraint and both are recorded in Section 1.5.

4.1.7 Design Rationale and Rejected Alternatives

The architecture above is one point in a space of possibilities, and the decisions that define it are collected here so that they can be assessed together. The alternatives are tabulated in Table 3.7 of Chapter 3; this section gives the reasoning for the three decisions that most shape the design.

Why the database and nothing else. Every mechanism the engine needs — an ordered append-only log, a queue with visibility timeouts, a timer store, and a linearizable compare-and-swap for ownership — is available within a single relational database, and crucially they are available *within one transaction*. Introducing a message broker for the queue or a coordination service for ownership would mean that a workflow decision and its consequences could no longer be committed atomically, and the engine would have to implement the missing atomicity itself. The dependency count would rise and the complexity would rise with it. This reasoning is what makes NFR-6 a design asset rather than an arbitrary restriction.

Why ownership is a lease rather than consensus. A full consensus protocol would remove the dependence on a single database as the arbiter of ownership. It would also require operating a second distributed system, which contradicts the project's central thesis. Because the fencing check makes a displaced node harmless regardless of what it believes, the weaker mechanism is sufficient for the safety property the system actu-

ally needs. Consensus would improve availability, not correctness, and availability is explicitly bounded rather than maximised in NFR-3.

Why compensation rather than locking. Holding database locks for the duration of a multi-step workflow would provide isolation between concurrent workflows, which sagas do not. AC/DC [**?**] measured the cost of that approach directly: with one-second steps their locking variant degraded to approximately two orders per second with 42% of orders failing, while compensation sustained sixteen orders per second with no failures. The isolation gap that sagas leave is real, and is acknowledged in Section 4.2.9 rather than solved.

4.2 Low Level Design

This section descends from the architecture to the mechanisms. It specifies the tables that hold every piece of durable state, the loop that executes a workflow, the device by which replay is matched to history, the concurrency handshake inside a worker, the failure analysis that bounds what the engine can promise, and the decision path of the semantic idempotency contribution.

4.2.1 Persistent Data Model

All durable state in the system lives in six tables, shown in Fig. 4.4. There is one PostgreSQL database for the whole cluster rather than one per shard: sharding is physically expressed as the `shard_id` column that every table carries, and what confines a node to its own shards is the lease it holds together with the `shard_id` present in every statement it issues.

The event log

Listing 4.3: The event log. The primary key provides append-only semantics without any explicit locking.

```
1 CREATE TABLE events (
2     shard_id    INT         NOT NULL,
3     workflow_id TEXT        NOT NULL,
4     run_id      UUID        NOT NULL,
```

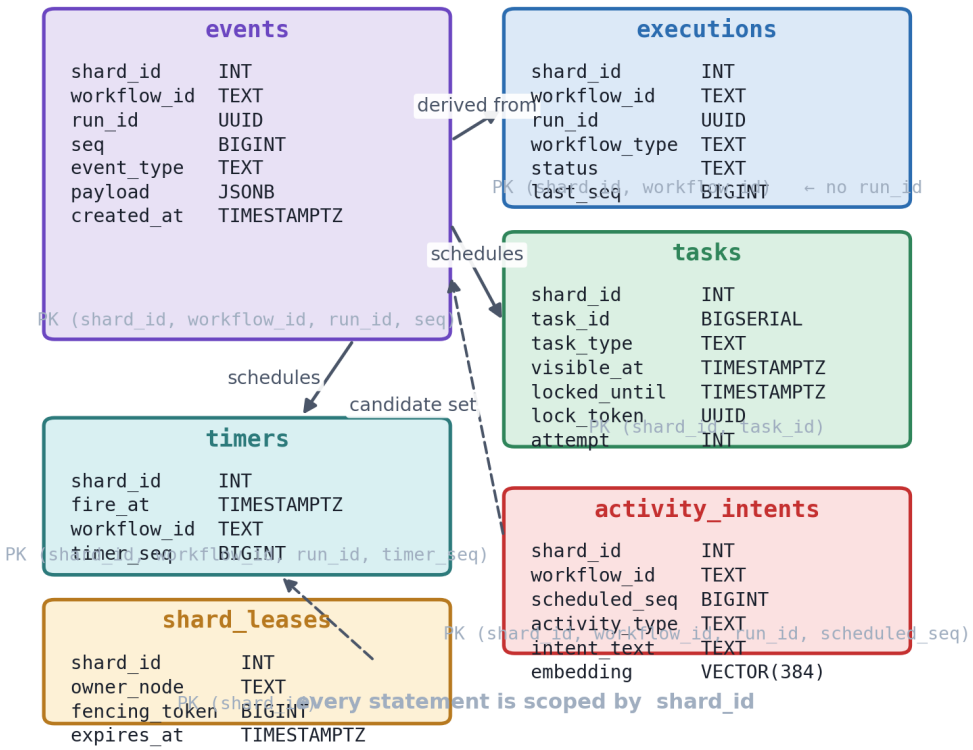


Fig. 4.4: The six tables holding every piece of durable state in the system. Every table carries `shard_id`, which is what sharding physically is in this design; no statement is ever issued without it.

```
    seq         BIGINT      NOT NULL,  -- 1, 2, 3, ... per (workflow_id,
        run_id)
    event_type  TEXT        NOT NULL,
    payload     JSONB       NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (shard_id, workflow_id, run_id, seq)
);
```

Three properties of Listing 4.3 are load-bearing rather than incidental.

The primary key provides append-only semantics for free. If two workers both attempt to append sequence number 47 for the same run, exactly one succeeds and the other receives a unique-constraint violation and rolls back. No lock is required to protect the log, because the primary key *is* the concurrency control. This is the cheapest correctness guarantee available in the whole design, and it is obtained at no cost.

The `run_id` column exists so that a workflow identifier can be re-run. A business key such as `order-8f3a` may be retried as a fresh execution after a terminal failure. The workflow identifier is the stable, user-facing business key; the run identifier denotes one attempt at it.

No `UPDATE` is ever issued against this table. The absence is checked by an automated assertion (FR-1), because an update to the log would mean that history could be rewritten, and every guarantee in this chapter would become conditional.

The execution pointer

The log is the truth, but scanning the whole log to answer "is this order still running?" would be prohibitive. The `executions` table therefore holds one mutable row per workflow, derived from the log, carrying the workflow type, its status and the highest sequence number written.

Its primary key is `(shard_id, workflow_id)` and deliberately excludes `run_id`. The effect is that at most one open run may exist per business key, which makes starting a workflow idempotent at the front door: a duplicate start request collides with the unique constraint and the engine returns the existing execution rather than creating a second

one. This is free deduplication obtained from the schema rather than from application logic.

Event types

4.2.2 The Workflow Task Loop

A *workflow task* means precisely one thing: *this workflow has new history; run its code forward until it blocks again*. The loop that processes one such task, shown in Fig. 4.5, is the heart of the engine.

Two steps deserve particular attention.

Step 8 is one transaction. The fencing-token verification, the insertion of every event implied by the decisions just made, the insertion of the corresponding task and timer rows, the update of the execution pointer, and the deletion of the workflow task row all commit together or not at all. Either the decision and every one of its consequences lands, or none of it does. There is no partial state to reconcile, ever, and no reconciliation code exists anywhere in the engine because none can be needed.

Step 9 is not an oversight. The in-memory workflow state is thrown away after every task. Correctness never depends on keeping it. This is what permits any node to pick up any workflow task at any moment without coordination, and it is what makes shard failover a matter of ownership rather than of state transfer.

4.2.3 Program Order as the Replay Join Key

Replay requires a way of matching "the call the code is making right now" to "the event that recorded that call last time". The mechanism is a counter, and it is the device most often missed in descriptions of durable execution.

Listing 4.4: Program order. The counter k is incremented once per call, in the order the code makes the calls, and appears nowhere in the source.

```
1 func (w *FoodDeliveryWorkflow) Execute(ctx WorkflowContext) error {
2     if err := ctx.ExecuteActivity("ChargePayment", w.order).Get(); err != nil
          {
3         return err                                                  // k = 0
4     }
```

Table 4.4: Event types written to the log, grouped by the mechanism that produces them

Event type	Group	Meaning
`WorkflowExecutionStarted`	Lifecycle	A run has begun; carries the workflow type and input
`WorkflowExecutionCompleted`	Lifecycle	Terminal success
`WorkflowExecutionFailed`	Lifecycle	Terminal failure
`ActivityTaskScheduled`	Activity	The intent to perform a side effect, recorded *before* it occurs; carries k and the dedup key
`ActivityTaskCompleted`	Activity	The side effect succeeded; carries its result
`ActivityTaskFailed`	Activity	The attempt failed; drives the retry policy
`TimerStarted`	Timer	A durable sleep has begun
`TimerFired`	Timer	The sleep has elapsed
`SignalReceived`	Signal	An external event was delivered into the workflow
`SideEffectRecorded`	Determinism	A non-deterministic value, computed once and journalled
`SemanticDedupEvaluated`	Idempotency	A deduplication verdict with its similarity, threshold and matched predecessor
`CompensationScheduled`	Saga	A compensating action has been scheduled
`CompensationCompleted`	Saga	A compensating action succeeded
`CompensationFailed`	Saga	A compensating action exhausted its retries
`CompensationResumed`	Saga	Compensation continued after a recovery

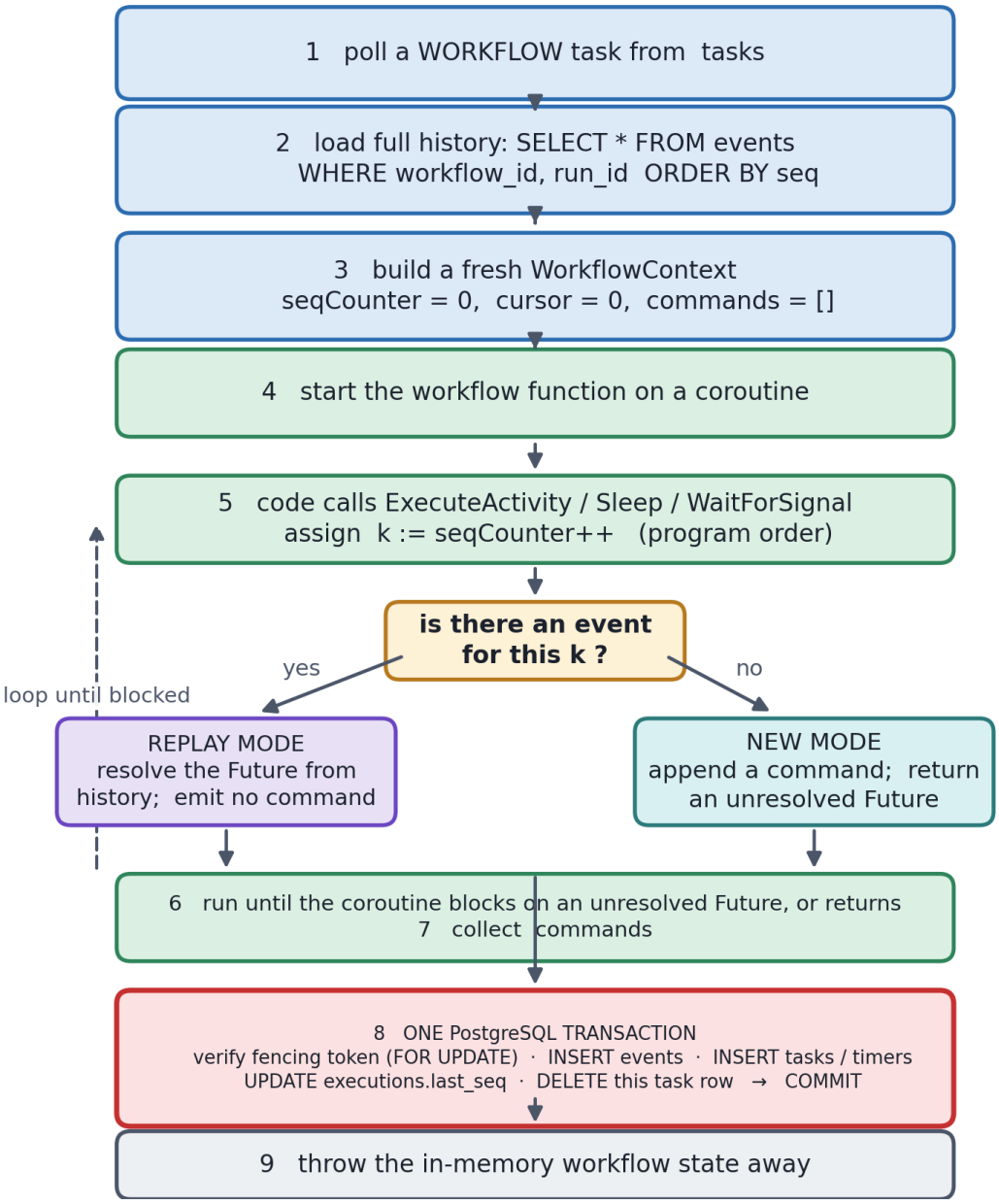


Fig. 4.5: The workflow task loop. Step 8 is a single transaction, which is what makes the system safe; step 9 discards the in-memory state, which is why correctness never depends on retaining it.

```
    if err := ctx.ExecuteActivity("NotifyRestaurant", w.order).Get(); err !=
        nil {
        return ctx.ExecuteActivity("RefundPayment", w.order).Get()    // k =
            1, 2
    }
    return nil
}
```

The value k is assigned by a counter incremented once per call, in the order the code makes the calls. It is recomputed identically on every replay because the code takes the same branches; the code takes the same branches because it is deterministic; and it is deterministic because it reads only from history. The value is written into the `ActivityTaskScheduled` payload, and on replay it is the join key between the live call and the recorded event.

If the code takes a different branch on replay — because it was edited, or because it consulted something non-deterministic — then k shifts, the match fails, and the engine raises a *non-determinism error*. The engine cannot repair this condition and does not attempt to. It has no way of knowing which version of the code told the truth, and any attempted reconciliation would corrupt the history silently. It therefore refuses to continue, which is the behaviour required by FR-3.

4.2.4 The Cooperative Scheduler

A call to `future.Get()` must suspend the workflow *without* blocking an operating-system thread and *without* handing scheduling control to the Go runtime in a way the engine cannot predict. What is required is cooperative, deterministic scheduling, obtained through the two-channel handshake in Listing 4.5.

Listing 4.5: The dispatcher–workflow handshake. Exactly one goroutine is runnable at any instant.

```
type coroutine struct {
    unblock chan struct{}   // dispatcher -> workflow: "keep going"
    yielded chan struct{}   // workflow -> dispatcher: "I am blocked"
}

// Called from workflow code, on the workflow goroutine.
func (f *future) Get() (any, error) {
```

```
    for !f.ready {
        f.co.yielded <- struct{}{}   // hand control to the dispatcher
        <-f.co.unblock               // wait to be resumed
    }
    return f.value, f.err
}

// Dispatcher, on the engine goroutine.
func (d *dispatcher) runUntilBlocked() {
    d.co.unblock <- struct{}{}   // let the workflow run
    <-d.co.yielded               // it ran until it blocked or finished
}
```

The two goroutines ping-pong, and exactly one is runnable at any instant. There is no true parallelism inside a workflow, and that is precisely the point: parallelism would be non-determinism. This also explains why `go func(){}`, `time.Sleep`, `sync.Mutex` and raw channels are forbidden inside workflow code — each hands scheduling to the Go runtime, which does not consult the event history.

4.2.5 Determinism in Practice

Determinism is a property of the workflow code, which the engine can detect violations of but cannot enforce in advance. Table 4.5 lists the constructs that break it and the replacement provided for each.

The escape hatch, `SideEffect`, is the general solution to unavoidable non-determinism, and is worth stating precisely because Section 4.2.10 is an application of it. In new mode the engine runs the supplied closure, obtains its value, appends a `SideEffectRecorded` event carrying k and that value, and returns it. In replay mode the engine does *not* run the closure; it reads the value out of the recorded event and returns that. Non-determinism is thereby neutralised by journalling its result.

The map-iteration entry deserves a specific warning. Go randomises map iteration order deliberately, so a loop that schedules activities while ranging over a map will schedule them in a different order on replay and produce a non-determinism error — intermittently, perhaps once in twenty runs. Sorting the keys is the fix.

Table 4.5: Constructs forbidden in workflow code, and their replacements

Forbidden	Why	Replacement
`time.Now()`	Differs on every replay	`ctx.Now()`, returning the timestamp recorded in the event log
`rand`, `uuid.New()`	Differs on every replay	`ctx.SideEffect(fn)`, which journals the result
`time.Sleep`	Blocks a real thread and depends on wall-clock time	`ctx.Sleep(d)`, a durable timer
`go func(){}`	Non-deterministic interleaving	`ctx.Go()` on the same cooperative scheduler
Map iteration	Go randomises map ordering deliberately	Sort the keys before iterating
Environment, files, network	The environment differs across machines	Perform it inside an activity
Global mutable state	Shared across replays	Pass it through the workflow input

4.2.6 A Worked Execution Trace

The mechanisms above are best understood through a concrete trace. Fig. 4.6 follows one workflow, `order-8f3a`, which hashes to shard 2, across its first three tasks.

Task 1 — the workflow starts. The gateway writes, in one transaction, two events — a workflow-started event at sequence 1 and a workflow-task-scheduled event at sequence 2 — together with a task row and an execution row. A workflow worker polls that task and sees a history of two events. It runs the code: the first call, `ExecuteActivity("ChargePayment")`, is assigned $k = 0$; no event for $k = 0$ exists in history, so the engine is in new mode; it emits a command and returns an unresolved future. The code calls `Get()` and blocks. The dispatcher regains control, finds no more history to feed, and the task is complete. The commit appends sequences 3 through 5 and inserts an activity task row.

Sequence 5 is the critical row of the whole trace, and Table 4.6 shows it. **The intent to charge has been durably recorded before anything was charged.** That ordering is the basis of every safety property the engine claims.

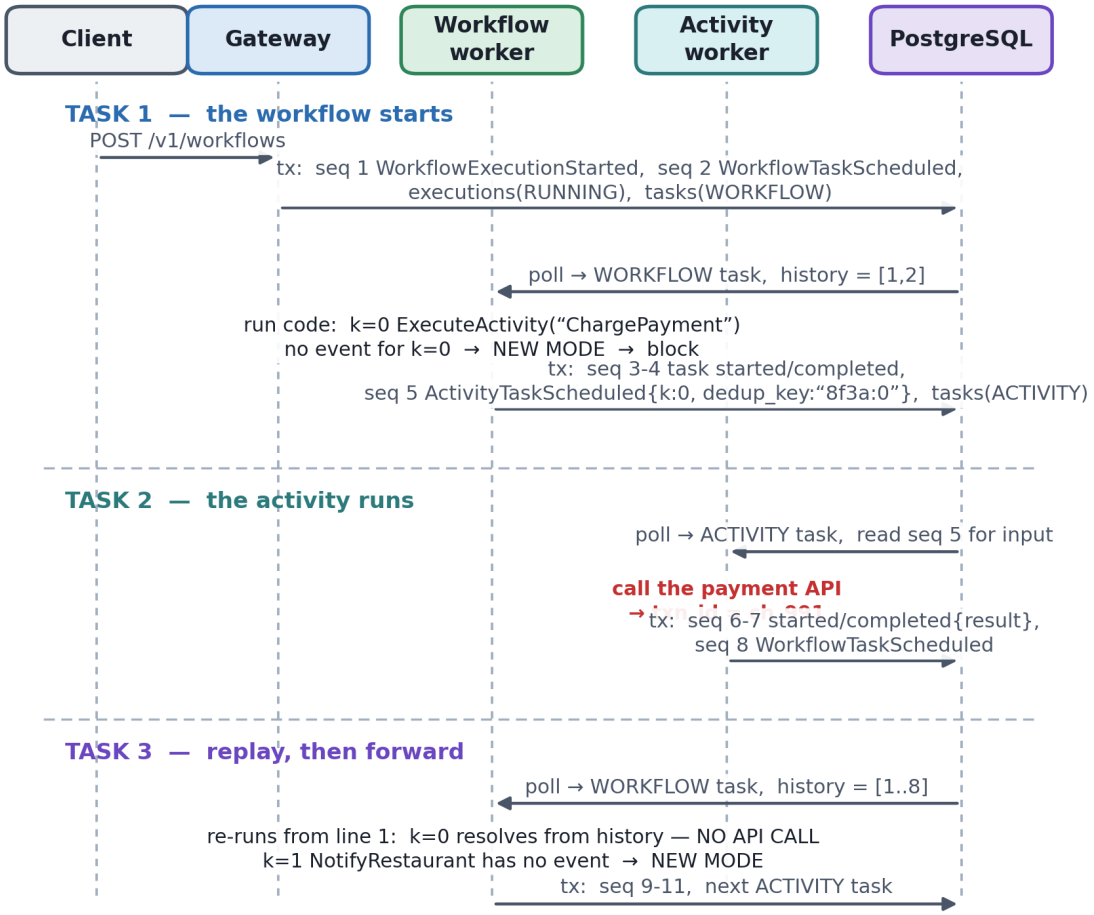


Fig. 4.6: A worked trace across three tasks. The crucial moment is the writing of sequence 5: the *intent* to charge is durably recorded before any money moves, which is the entire basis of the engine's safety.

Table 4.6: The events written when the first workflow task commits

seq	event_type	payload
3	`WorkflowTaskStarted`	`{"worker":"node-a"}`
4	`WorkflowTaskCompleted`	`{}`
5	`ActivityTaskScheduled`	`{"k":0, "activity":"ChargePayment", "input":{…}, "dedup_key":"8f3a:0"}`

Task 2 — the activity runs. An activity worker polls the activity task, reads sequence 5 for its input, and calls the payment API, which succeeds. The worker appends `ActivityTaskStarted`, then `ActivityTaskCompleted` carrying the result, then a fresh `WorkflowTaskScheduled`; it deletes the activity task row and inserts a workflow task row. The result is now in history and will never be recomputed.

Task 3 — replay, then forward. A worker polls the workflow task and sees a history of eight events. *It runs the workflow function from the very first line again.* The call at $k = 0$ finds both `ActivityTaskScheduled` and `ActivityTaskCompleted` in history, so the engine is in replay mode: the future resolves immediately with the recorded result and **no API call is made**. The code never knows the difference. Execution continues past the point previously reached, the call at $k = 1$ finds nothing in history, and the engine returns to new mode.

That is the complete engine. Everything remaining in this section makes it survive progressively worse failures.

4.2.7 The Durable Task Queue

Work is dispatched through a table rather than a message broker. The table carries a visibility timestamp, a lock expiry, a lock token and an attempt counter. Critically it is a *lease queue* rather than a delete-on-read queue: a worker does not remove the row, it takes a time-boxed lock on it. If the worker dies, the lock expires and the task becomes visible again. That is the entire crash-recovery story for in-flight tasks, and it requires no detection mechanism and no timeout thread.

Listing 4.6: The poll query. `SKIP LOCKED` is what allows many workers to dequeue concurrently without serialising behind a single row lock.

```
UPDATE tasks
SET    locked_until = now() + interval '30 seconds',
       lock_token   = gen_random_uuid(),
       attempt      = attempt + 1
WHERE  (shard_id, task_id) IN (
    SELECT shard_id, task_id FROM tasks
    WHERE  shard_id = $1
      AND  task_type = $2
      AND  visible_at <= now()
      AND  (locked_until IS NULL OR locked_until < now())
```

```
    ORDER BY task_id
    FOR UPDATE SKIP LOCKED
    LIMIT 1
)
RETURNING *;
```

The `SKIP LOCKED` clause in Listing 4.6 is not an optimisation. Without it, every worker attempting to poll would queue behind the same row lock and the system would serialise regardless of how many workers were running; with it, each worker skips rows already locked by another and the queue scales with worker count. This single clause is the reason the design can satisfy NFR-6 without an external broker.

Delayed retries require no additional mechanism: a retry is an ordinary row whose `visible_at` is set to a future time. The queue already understands delay.

4.2.8 Durable Timers and Signals

Timers

A call to `ctx.Sleep(5 * time.Minute)` at $k = 3$ proceeds as follows. In new mode the engine appends `TimerStarted` carrying k and the firing time, inserts a row into the timer table, and returns an unresolved future; the workflow blocks and the worker moves on to other work. **Nothing holds a thread, a connection or a byte of memory for those five minutes.** A background scanner per shard polls for timers whose firing time has passed, and for each one appends `TimerFired`, deletes the timer row and inserts a workflow task — all in one transaction. The workflow task then replays from the first event, finds both `TimerStarted` and `TimerFired` at $k = 3$, resolves the future immediately and continues.

A five-minute sleep and a five-day sleep therefore cost exactly the same, and a cluster restart during the wait is invisible, because the timer is a row rather than a thread.

Racing a timer against an activity

A common requirement — "wait up to five minutes for the restaurant to respond" — is expressed as a selector over two futures. The selector must resolve **in history order**, not in whichever-fires-first order. Resolving by arrival would mean the branch taken during the live run could differ from the branch taken during replay, which is precisely

a non-determinism error. The engine therefore resolves a selector by scanning history for whichever completion event carries the lower sequence number.

Signals

A signal is an external event injected into a running workflow: a restaurant accepting an order, or a human approving an agent's plan. On receipt the engine routes to the owning shard, verifies the fencing token, appends `SignalReceived`, inserts a workflow task, and commits. The signal is durable *before* any workflow code observes it, so if the entire cluster dies one millisecond later the signal is not lost (FR-11).

Signals that arrive before the workflow has reached the corresponding wait must still be delivered. They are buffered in history and matched by name rather than by arrival order relative to k. Failing to do this is the classic defect in hand-rolled implementations of this mechanism.

4.2.9 Retry Semantics and the Crash-Window Analysis

This subsection contains the engine's most important honest claim, and it is stated before the evidence rather than after it:

> **The engine provides at-least-once execution of activities and exactly-once observation of their results.** True exactly-once execution across a process boundary is impossible; the engine's role is to make the boundary narrow and to hand the application the tool with which to close it.

The proof is a catalogue of every place a crash can land during the execution of one activity. Fig. 4.7 shows the four windows on a timeline and Table 4.7 gives them in full.

Window C is irreducible. The side effect and the record of it live in two different systems, and there is no distributed transaction spanning the engine's database and a third-party payment processor. The same conclusion is reached independently by DBOS [**?**] and R3 [5]. What the engine does about it is threefold:

1. It derives a stable **deduplication key** — `"8f3a:0"` — from the workflow identifier and the deterministic k. The key is identical on every retry, because k is stable across replays.

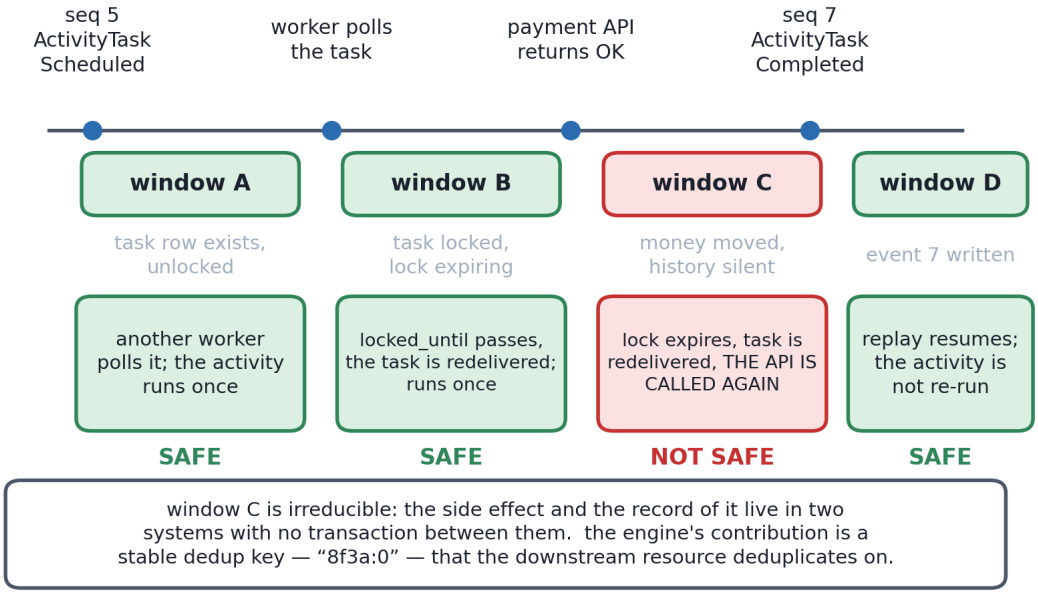


Fig. 4.7: Every place a crash can land during one activity. Three of the four windows recover safely without intervention. Window C is irreducible, and the design says so rather than claiming exactly-once execution.

Table 4.7: Crash-window analysis for a single activity execution

	Crash point	State on disk	On recovery	Safe?
A	After the activity is scheduled, before any worker polls	Task row exists, unlocked	Another worker polls it; the activity runs once	Yes
B	Worker polled the task, crashed before calling the API	Task locked, lock expiring	The lock expires, the task is redelivered, the activity runs once	Yes
C	**The API succeeded; the worker crashed before writing the completion event**	Task locked; money has moved; history is silent	The lock expires, the task is redelivered, **and the worker calls the API a second time**	**No**
D	The completion event was written; crash before the workflow task is processed	Workflow task row exists	Redelivered; replay resumes; the activity is not re-run	Yes

2. It passes that key to the activity, which forwards it as the downstream service's idempotency key.
3. The downstream service performs the actual deduplication.

The engine's contribution is therefore precisely *generating a key that is stable across crashes, retries and replays*, which the application could not have generated on its own, because the application does not know its own program order. The final deduplication happens where it must, at the resource. Being explicit about this division is a strength of the design rather than a concession.

Table 4.8: Retry policy parameters, evaluated by the workflow worker on seeing an activity failure

Parameter	Default	Effect
`InitialInterval`	1 s	Delay before the first retry
`BackoffCoefficient`	2.0	Multiplier applied to each successive delay
`MaximumInterval`	100 s	Ceiling on the computed delay
`MaximumAttempts`	5	Zero means unlimited
`NonRetryableErrors`	empty	Error types that terminate immediately, such as an invalid card, for which retrying cannot succeed

4.2.10 Saga Compensation

Compensation is not a separate engine subsystem. It is a workflow pattern that the engine makes durable, and understanding that is the point of this subsection.

Listing 4.7: Saga compensation. Each compensation is registered only *after* the forward step it undoes has succeeded.

```
1 func (w *FoodDeliveryWorkflow) Execute(ctx WorkflowContext) error {
2     saga := NewSaga(ctx)
3     defer saga.Cleanup()
4
5     if err := ctx.ExecuteActivity("ChargePayment", w.order).Get(); err != nil
          {
6         return err                                  // nothing done, nothing
              to undo
```

```
    }
    saga.AddCompensation("RefundPayment", w.order) // registered AFTER success

    if err := ctx.ExecuteActivity("ReserveInventory", w.order).Get(); err !=
        nil {
        return saga.Compensate()                  // -> RefundPayment
    }
    saga.AddCompensation("ReleaseInventory", w.order)

    if err := ctx.ExecuteActivity("NotifyRestaurant", w.order).Get(); err !=
        nil {
        return saga.Compensate()   // -> ReleaseInventory, then RefundPayment
    }
    return nil
}
```

Three mechanical properties, illustrated in Fig. 4.8, follow from Listing 4.7.

The compensation stack is not journalled. It does not need to be. The stack is rebuilt identically on every replay, because the code is deterministic and the successes it depends upon are already in history. Journalling it would duplicate state that replay already provides, which is always a signal that the governing principle has been misunderstood.

Compensation pops in last-in-first-out order, and each compensating action runs as an ordinary activity — journalled, retried, supplied with a dedup key, and crash-safe exactly like any other.

Registration happens after the forward step succeeds. Registering before would mean that a failed charge triggered a refund of money that was never taken.

Three hard cases are raised here rather than left to be discovered.

A compensation can itself fail. It retries with backoff like any activity. If it exhausts its retries, the workflow enters `COMPENSATING_FAILED` and requires human intervention (FR-13). There is no automatic resolution, and the original saga formulation says as much [25]. Surfacing the state in the administrative interface is the correct response.

Some steps cannot be compensated. A sent email cannot be unsent. The saga's guarantee is *semantic* rollback — a cancellation message — not *physical* rollback.

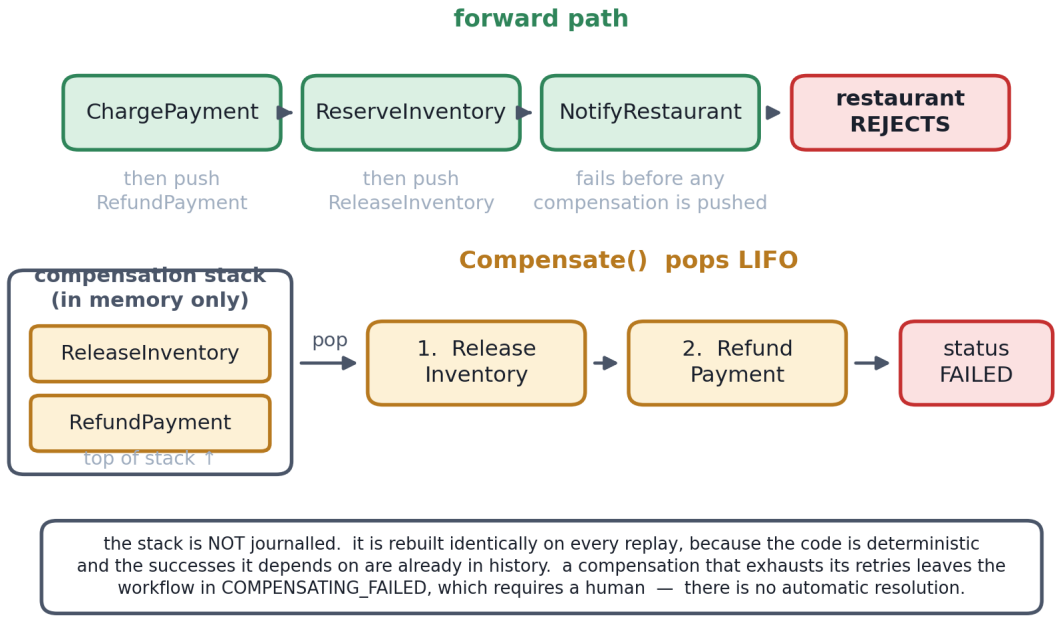


Fig. 4.8: Saga compensation. The stack exists only in memory and is rebuilt by replay; compensations run in reverse order as ordinary activities; and a compensation that exhausts its retries leaves the workflow in a terminal state requiring human intervention.

Sagas provide no isolation. Between a step and its compensation, other workflows can observe the intermediate state. Sagas are ACD rather than ACID. Daraghmi et al. [27] attack exactly this gap with a quota cache; this project accepts the limitation rather than solving it, and says so.

4.2.11 The Semantic Idempotency Decision Path

This subsection specifies the research contribution of the project.

The precise failure being addressed

A framing correction is necessary first, because a loose statement of this problem is easy to refute. When the *engine* retries a scheduled activity, the arguments are already frozen in the `ActivityTaskScheduled` event and are byte-identical on every attempt. Exact matching handles that case completely. An engine-level retry is therefore *never* the source of a re-worded duplicate.

The real source is **agent re-planning**, and the sequence that produces it is the one shown earlier in Fig. 2.2. A plan is journalled; a tool call executes and books a flight; a second planning step begins and the process crashes before that step's result is journalled. On recovery, replay resolves the first plan and the tool call from history without re-running them, but the second planning step genuinely re-executes. At a non-zero sampling temperature it returns a different plan, which re-issues the same intent in different words. The exact-match hash equals no prior key, the action executes, and the flight is booked twice.

The claim is therefore stated narrowly and defensibly: *agent re-planning, whether triggered by a crash or by a tool failure, produces semantically duplicate tool calls; the workflow's event history is the natural place to detect them; and no durable execution engine looks there.*

Where the verdict must be computed, and why

The naive placement is inside the activity worker, immediately before the tool is called. **That placement is wrong, and understanding why is the strongest statement this design can make about itself.**

The similarity check is non-deterministic. It depends on an embedding model, a floating-point threshold and a candidate set. Were the verdict computed inside the activity worker, then on replay it would either be recomputed — possibly differently, causing history to diverge — or skipped entirely, meaning the workflow's decision would not be reproducible. Either outcome breaks the governing principle of Section 4.1.1.

The correct placement follows directly: **the verdict is computed once, on the workflow path, and journalled as an event.** It is the `SideEffect` pattern of Section 4.2.5 applied to a deduplication decision. Fig. 4.9 shows both modes, and Listing 4.8 gives them step by step.

Listing 4.8: The decision path in both modes. The asymmetry between them is the entire mechanism.

```
ExecuteActivity("book_flight", args)   mode = Semantic,   at k = 7

NEW MODE:
```

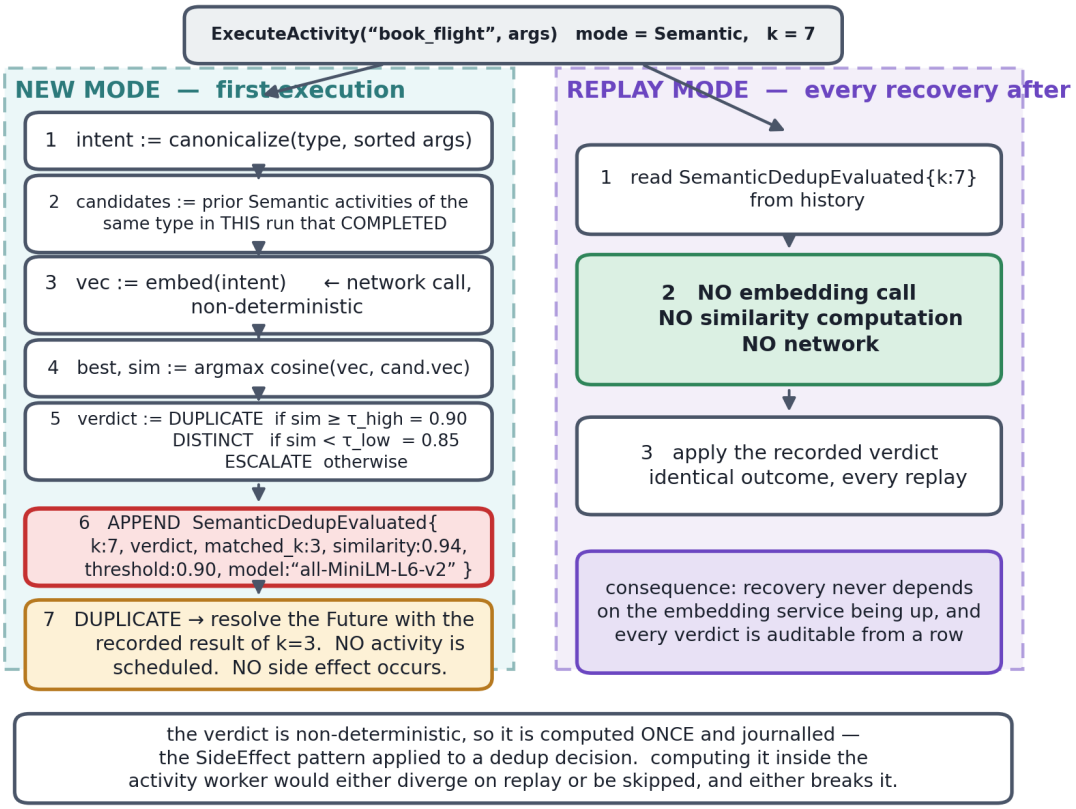


Fig. 4.9: The semantic idempotency decision path. The non-deterministic verdict is computed once in new mode and journalled; every subsequent replay reads the recorded verdict and performs no embedding call, no similarity computation and no network access.

```
  1. intent     := canonicalize("book_flight", sorted normalized args)
  2. candidates := prior Semantic activities of the SAME type in THIS
                   run's history that COMPLETED successfully
  3. vec        := embed(intent)             -- network, non-deterministic
  4. best, sim  := argmax cosine(vec, candidate.vec)
  5. verdict    := DUPLICATE  if sim >= tau_high (0.90)
                   DISTINCT   if sim <  tau_low  (0.85)
                   ESCALATE   otherwise
  6. APPEND  SemanticDedupEvaluated{ k:7, verdict:"DUPLICATE", matched_k:3,
             similarity:0.94, threshold:0.90, model:"all-MiniLM-L6-v2" }
  7. DUPLICATE -> resolve the Future with the recorded result of k=3.
                  NO activity is scheduled.  NO side effect occurs.
     DISTINCT  -> emit ScheduleActivity as normal.
     ESCALATE  -> emit a human-approval signal wait, or execute
         (configurable).

REPLAY MODE:
  1. Read SemanticDedupEvaluated{k:7} from history.
  2. NO embedding call.  NO similarity computation.  NO network.
  3. Apply the recorded verdict.  Identical outcome, every replay, forever.
```

Three consequences follow and are worth stating explicitly.

Replay is fast and offline. No embedding service is needed in order to recover a workflow. If the embedding provider is unavailable, in-flight workflows still recover normally. A design that called the model on every replay would have coupled recovery availability to a third-party API, which is the kind of dependency this project exists to avoid. This satisfies constraint C-4.

The verdict is auditable. The stored event carries the similarity, the threshold, the matched predecessor and the model identifier. When somebody asks why a particular flight was never booked, the answer is a row rather than a reconstruction. For a mechanism empowered to suppress real-world actions, that audit trail is not a convenience; it is what makes the feature deployable (NFR-10, FR-15).

It composes with the existing determinism story rather than fighting it, because it is an instance of a mechanism the engine already had.

Storage and the candidate set

Intents are stored in a dedicated table keyed by shard, workflow, run and scheduled sequence, carrying the canonicalised intent text and a 384-dimensional embedding vector held in a `VECTOR(384)` column.

Candidate sets are per-workflow-run and small — tens of rows. Brute-force cosine similarity over a set that size takes microseconds, and no approximate nearest-neighbour index is required. This is worth stating explicitly, because it pre-empts the obvious scaling objection: the search is bounded by the length of one workflow's history, not by the size of a global corpus. That is a genuine architectural advantage over semantic caching systems [30], which must search a corpus shared by all users.

The construction of the intent text matters more than the choice of model. The canonical form is the activity type followed by its sorted, normalised arguments. For an activity such as `search_flights` whose argument is itself natural language, this embeds meaningfully. For an activity such as `charge_payment` whose argument is an amount, it does not — which is precisely why the idempotency mode is opt-in per activity and defaults to exact matching.

Failure modes, stated first

Table 4.9: Failure modes of the semantic comparator. The asymmetry between them is the design's central constraint.

	Meaning	Cost	Mitigation
False positive	Two genuinely different calls are judged duplicate, so a real action is silently skipped	**Severe and asymmetric.** A flight the user wanted is never booked, and nothing reports an error	Opt-in per activity; high τ; escalation band; every verdict journalled and visible in the administrative panel
False negative	A true duplicate is not caught and executes twice	Equal to the exact-match baseline. No regression	None required

The asymmetry in Table 4.9 is the honest core of this contribution and is presented as a designed-for constraint rather than left to be discovered: **the mechanism is strictly better than the baseline on false negatives and strictly worse on false positives, and it is therefore opt-in, conservatively thresholded and fully audited.** Claiming the mechanism is free is what would invite the design to be picked apart.

The two-threshold structure, with an escalation band between τ_{low} and τ_{high}, is taken from work on calibration in semantic caching [**?**], which shows that a single threshold forces a binary decision at exactly the point where the model is least confident. The difficulty of calibrating such a threshold at all is not a shortcoming unique to this project: vCache [**?**] reports the same finding across approximately 45,000 examples, with problematic cut-offs falling in the same range this design uses.

Finally, the relationship to the closest competing work bears restating. Robust Agent Compensation [**?**] undoes an agent's actions after a failure is detected. This mechanism prevents the duplicate from executing. A duplicate that succeeds raises no failure and therefore triggers no compensation, which is precisely the case this design catches and RAC cannot.

4.2.12 Node Runtime Composition

The design concludes with what actually runs on a machine. One Go binary, one process per node, with the goroutine structure shown in Fig. 4.10: an HTTP server, a lease manager, a metrics collector, and — for each shard the node currently owns — one timer scanner, four workflow workers and eight activity workers.

Each worker goroutine is a plain loop: poll, do the work, commit, repeat, with exponential backoff from ten milliseconds to one second when a poll returns nothing, so that an idle cluster does not saturate the database.

The shutdown path deserves a closing observation, because it demonstrates the economy the fencing design buys. Every worker verifies the fencing token inside its own commit. If a shard is lost while work is in flight, its workers discover this when their own transaction fails, abandon it and exit. **No coordination is required to shut them down, and no supervisor needs to notify them — the database tells them.** A mechanism designed for correctness under split-brain turns out to supply orderly shutdown

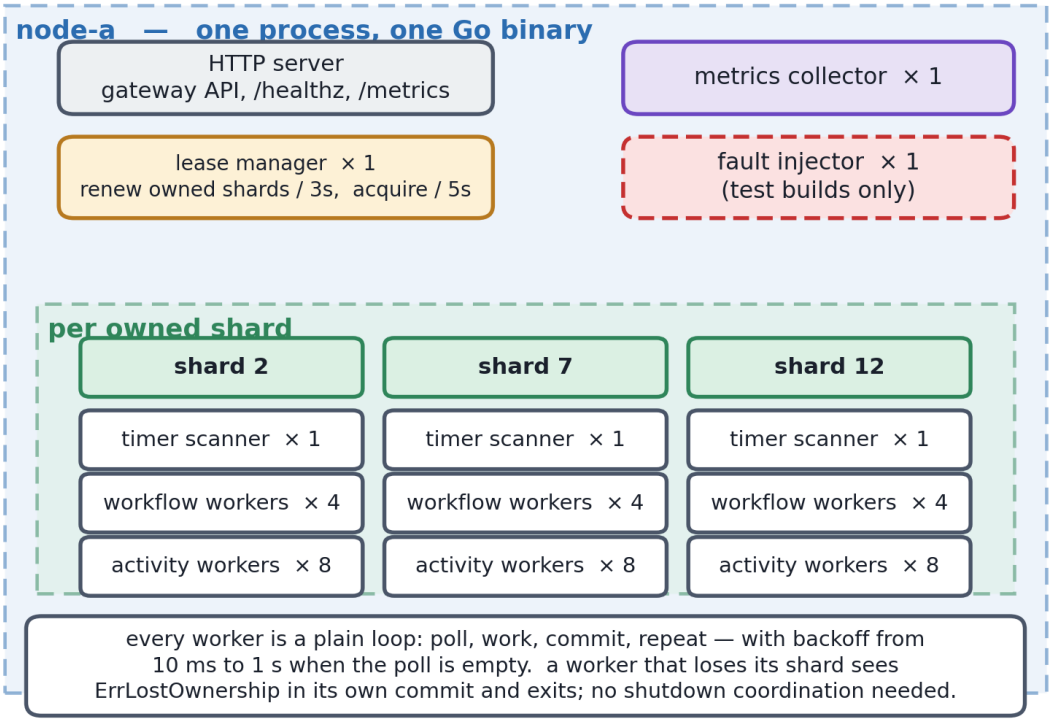


Fig. 4.10: The goroutine composition of one node. Worker counts are per owned shard, so a node's concurrency scales with the number of shards it owns rather than with cluster size.

as a side effect, which is a reasonable indication that the abstraction was drawn in the right place.

Table 4.10: Runtime configuration parameters and their defaults

Parameter	Default	Meaning
`ShardCount`	16	Fixed partition count; see Table 4.3
`LeaseTTL`	10 s	Lease duration, bounding failover time
`LeaseRenew`	3 s	Renewal interval, giving three attempts per TTL
`TaskLock`	30 s	Visibility timeout on a claimed task; must exceed the longest expected activity
`WorkflowWorkers`	4	Workflow worker goroutines per owned shard
`ActivityWorkers`	8	Activity worker goroutines per owned shard; higher than the workflow count because activities block on external I/O
`SyncCommit`	true	Synchronous replication, trading throughput for a zero loss window
`TauHigh`	0.90	Similarity at or above which a call is a duplicate
`TauLow`	0.85	Similarity below which a call is distinct; the band between the two escalates

The design presented in this chapter is complete to the level at which it can be implemented. Chapter 5 describes that implementation, Chapter 6 the testing strategy, and Chapter 7 the measured results of running the engine under deliberately induced failure.

REFERENCES

[1] Temporal Technologies, "Temporal platform documentation," [Online]. Available: `https://docs.temporal.io/`. Accessed: Oct. 2026.

[2] Restate, "What is durable execution?" [Online]. Available: `https://restate.dev/what-is-durable-execution/`. Accessed: Oct. 2026.

[3] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon and C. S. Meiklejohn, "Durable functions: Semantics for stateful serverless," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Art. 133, 2021.

[4] A. Skiadopoulos, Q. Li, P. Kraft, K. Kaffes, D. Hong, S. Mathew, D. Bestor, M. Cafarella, V. Gadepally, G. Graefe, J. Kepner, C. Kozyrakis, T. Kraska, M. Stonebraker, L. Suresh and M. Zaharia, "DBOS: A DBMS-oriented operating system," *Proc. VLDB Endow.*, vol. 15, no. 1, pp. 21–30, 2022.

[5] Q. Li, P. Kraft, M. Cafarella, C. Demiralp, G. Graefe, C. Kozyrakis, M. Stonebraker, L. Suresh, X. Yu and M. Zaharia, "R3: Record-replay-retroaction for database-backed applications," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 3085–3097, 2023.

[6] S. Lee, Z. Guo, O. Sunercan, J. Ying, T. Kooburat, S. Biswal, J. Chen, K. Huang, Y. Cheung, Y. Zhou, K. Veeraraghavan, B. Damani, P. M. Ruiz, V. Mehta and C. Tang, "Shard Manager: A generic shard management framework for geo-distributed applications," in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP)*, 2021, pp. 553–569.

[7] P. Hunt, M. Konar, F. P. Junqueira and B. Reed, "ZooKeeper: Wait-free coordination for internet-scale systems," in *Proc. USENIX Annual Technical Conf.*, 2010, pp. 145–158.

[8] C. Gray and D. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proc. 12th ACM Symp. Operating Systems Principles (SOSP)*, 1989, pp. 202–210.

[9] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[10] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conf.*, 2014, pp. 305–319.

[11] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez and I. Stoica, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.

[12] R. S. S. Kumar, P. Bryan, C. Seifert, N. Chawla, A. Kumar and Y. Zunger, "Taxonomy of failure modes in agentic AI systems," Microsoft AI Red Team, White paper, Apr. 2025.

[13] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[14] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann, 1992.

[15] J. Kreps, N. Narkhede and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[16] J. Goldstein, A. Abdelhamid, M. Barnett, S. Burckhardt, B. Chandramouli, D. Gehring, N. Lebeck, C. Meiklejohn, U. F. Minhas, R. Newton, R. A. Peshawaria, T. Zaccai and I. Zhang, "A|MBROSIA: Providing performant virtual resiliency for distributed applications," *Proc. VLDB Endow.*, vol. 13, no. 5, pp. 588–601, 2020.

[17] H. Zhang, A. Cardoza, P. B. Chen, S. Angel and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 1187–1204.

[18] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP)*, 2021, pp. 691–707.

[19] S. Burckhardt, B. Chandramouli, C. Gillum, D. Justo, K. Kallas, C. McMahon, C. S. Meiklejohn and X. Zhu, "Netherite: Efficient execution of serverless workflows," *Proc. VLDB Endow.*, vol. 15, no. 8, pp. 1591–1604, 2022.

[20] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine and D. Lewin, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web," in *Proc. 29th ACM Symp. Theory of Computing (STOC)*, 1997, pp. 654–663.

[21] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205–220.

[22] A. Adya, D. Myers, J. Howell, J. Elson, C. Meek, V. Khemani, S. Fulger, P. Gu, L. Bhuvanagiri, J. Hunter, R. Peon, L. Kai, A. Shraer, A. Merchant and K. Lev-Ari, "Slicer: Auto-sharding for datacenter applications," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 739–753.

[23] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2006, pp. 335–350.

[24] M. J. Fischer, N. A. Lynch and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. ACM*, vol. 32, no. 2, pp. 374–382, 1985.

[25] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1987, pp. 249–259.

[26] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. 3rd Conf. Innovative Data Systems Research (CIDR)*, 2007, pp. 132–141.

[27] E. Y. Daraghmi, C.-P. Zhang and S.-M. Yuan, "Enhancing saga pattern for distributed transactions within a microservices architecture," *Applied Sciences*, vol. 12, no. 12, Art. 6242, 2022.

[28] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[29] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[30] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proc. 3rd Workshop for NLP Open Source Software (NLP-OSS)*, 2023, pp. 212–218.

[31] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar and M. A. Gulzar, "MeanCache: User-centric semantic caching for LLM web services," in *Proc. IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2025.

[32] L. G. Schroeder, S. Liu, A. Cuadron, M. Zhao, S. Krusche, A. Kemper, M. Zaharia and J. E. Gonzalez, "vCache: Verified semantic prompt caching," arXiv:2502.03771, 2025.

[33] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th European Conf. Computer Systems (EuroSys)*, 2015, Art. 18.

[34] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[35] B. Schroeder, A. Wierman and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2006, pp. 239–252.

[36] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[37] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[38] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation," [Online]. Available: `https://www.postgresql.org/docs/16/`. Accessed: Oct. 2026.